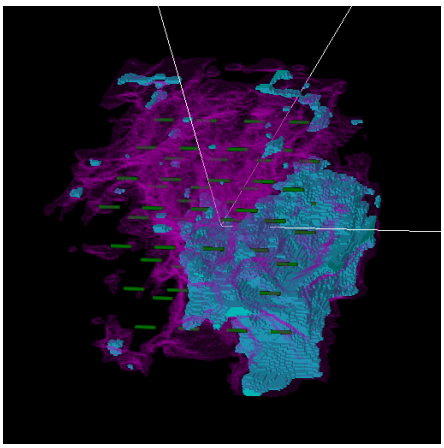
User manual for `egs_brachy`

A versatile and fast EGSnrc application for brachytherapy

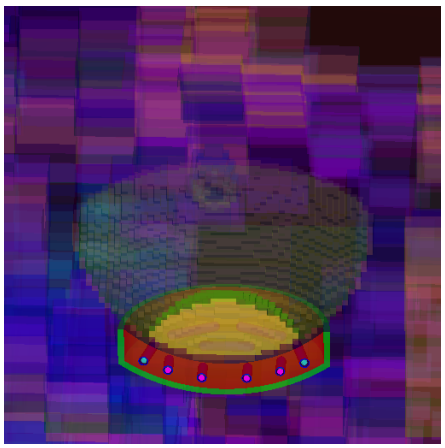
Rowan M. Thomson, Randle E. P. Taylor, Marc J. P. Chamberland,
and D. W. O. Rogers
Carleton Laboratory for Radiotherapy Physics, Department of Physics,
Carleton University, Ottawa, Ontario, K1S 5B6, Canada

Last edited 2026/10/19 at 13:04:04
Latest version available at
https://clrp-code.github.io/egs_brachy/pdf/egs_brachy_user_manual.pdf

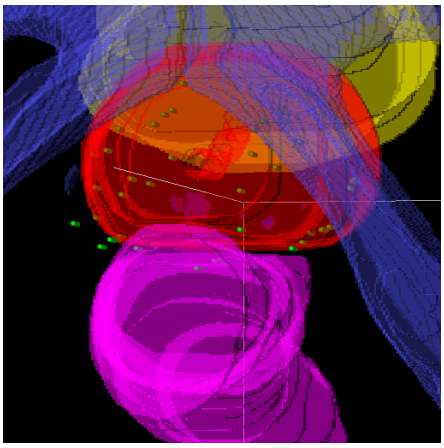
Report CLRP-17-02



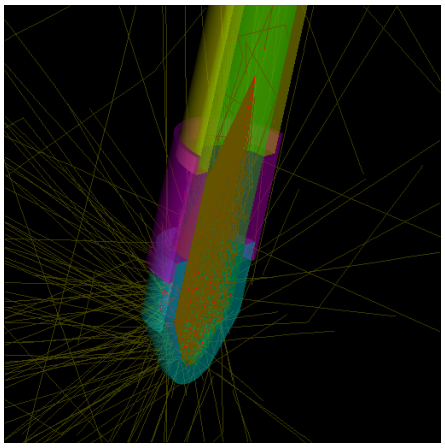
^{103}Pd breast implant



^{125}I eye plaque



^{125}I prostate implant



Electronic brachytherapy

Contents

1. Introduction **1**

2. Licence **3**

3. Installation **3**
3.1. Installing from a release tarball . . . 3
3.2. Git checkout . . . 4
3.3. Updating `egs_brachy` . . . 4

4. Running egs_brachy **5**

5. Brief description of `egs_brachy` input files **5**
5.1. `run control` . . . 6
5.2. `run mode` . . . 6
5.3. `media definition` . . . 6
5.4. `geometry definition` . . . 6
5.4.1. `egs_autoenvelope` . . . 9
5.5. `source definition` . . . 10
5.6. `volume correction` . . . 11
5.7. `scoring options` . . . 12
5.8. `variance reduction` . . . 14
5.9. `MC transport parameters` . . . 14

6. Brief description of `egs_brachy` outputs **15**
6.1. The `.egslog` output . . . 15
6.2. `.3ddose` and other outputs . . . 16

7. Description of egs_brachy **16**
7.1. Radiation transport modelling . . . 16
7.2. Geometry and source modelling . . . 17
7.3. Run modes . . . 18
7.4. Scoring options . . . 19
7.4.1. Dose . . . 19
7.4.2. Calculation of absolute dose from doses output by `egs_brachy` . . . 22
7.4.3. Phase space . . . 23
7.4.4. Spectrum . . . 24
7.5. Features to enhance simulation efficiency . . . 24
7.5.1. Recycling of photons emitted from sources . . . 24
7.5.2. Use of phase-space source . . . 25

7.5.3. Features for electronic brachytherapy sources . . . 25

8. Data distributed with egs_brachy 26
8.1. Material data definitions distributed with egs_brachy . . . 26
8.2. Initial radionuclide spectra . . . 30

9. Geometries distributed with egs_brachy 30
9.1. Sources . . . 31
9.2. Phantoms . . . 31
9.3. Eye plaques . . . 32
9.4. Applicators . . . 32

10. Example input files distributed with egs_brachy 33
10.1. ex_single_source.egsinp . . . 33
10.2. ex_single_source_phase_space_and_spectrum_scoring.egsinp . . . 33
10.3. ex_single_source_from_phase_space.egsinp . . . 33
10.4. ex_prostate_permanent_implant.egsinp . . . 34
10.5. ex_prostate_permanent_implant_pegs4.egsinp . . . 34
10.6. ex_prostate_permanent_implant_TG43.egsinp . . . 34
10.7. ex_mbdca-wg_center.egsinp, ex_mbdca-wg_center_water_only.egsinp, and ex_mbdca-wg_x7cm.egsinp . . . 34
10.8. ex_hdr_source_with_applicator.egsinp and ex_hdr_source_with_applicator_create_volume_correction.egsinp 35
10.9. ex_PeppaBreastHDR192Ir.egsinp . . . 35
10.10 ex_xray_sph.egsinp . . . 35
10.11 ex_brem_cyl.egsinp . . . 35

11. Helpful hints for egs_brachy simulations 36
11.1. Geometry preview . . . 36
11.2. Examining the .egslog output . . . 36
11.3. Run modes and workflow . . . 36
11.4. Source and geometry transformations . . . 37
11.5. Geometry errors . . . 37
11.6. Use of a wrapper geometry about sources . . . 38
11.7. Particle recycling . . . 38
11.8. Phase-space sources . . . 38
11.9. nbatch and nchunk options . . . 38
11.10 Source orientation . . . 39

12. Acknowledgements 39

References 41

Index 44

1. Introduction

egs_brachy is a versatile and fast EGSnrc application designed for brachytherapy, employing the egs++ library for modelling geometries and sources, developed by members of the Carleton Laboratory for Radiotherapy Physics (CLRP) within the Department of Physics at Carleton University. The current document serves as a user manual for egs_brachy to complement the original 2016 egs_brachy paper [1] that provides a general overview of the code, details on egs_brachy benchmarking and characterization of simulation efficiency, and some sample calculation times for clinical scenarios. This user manual frequently refers to the html technical reference documentation (which accompanies the egs_brachy code distribution or can be found online at https://clrp-code.github.io/egs_brachy/pdf/egs_brachy_user_manual.pdf/index.html or the pdf version of the same technical reference manual (>500 pages) at https://clrp-code.github.io/egs_brachy/pdf/egs_brachy_user_manual.pdf/egs_brachy_technical_manual.pdf) for details regarding input file specifications etc.

CLRP also distributes eb_gui, a Qt-based graphical user interface for egs_brachy . eb_gui assists with setting up egs_brachy calculations and provides tools for converting to and from DICOM format and analysing .3ddose files. Installation instructions and a user guide (eb_gui_UserGuide.pdf) are available in the eb_gui repository at https://github.com/clrp-code/eb_gui.

An overview of this user manual is as follows: Section 2. describes egs_brachy licence and copyright. Installation of egs_brachy is described in section 3.: release tarballs and eb-setup.sh (section 3.1.), git checkout for developers (section 3.2.), and updating an existing install (section 3.3.). Running egs_brachy is described in section 4., followed by an overview of egs_brachy input and output files in sections 5. and 6., respectively. A more detailed description of egs_brachy, to complement our initial paper [1], appears in section 7., including radiation transport modelling (section 7.1.), geometry and source modelling (section 7.2.), run modes (section 7.3.), scoring options (section 7.4.; includes a description in section 7.4.2. of the calculation of absolute doses from doses output by egs_brachy), and features to enhance simulation efficiency (section 7.5.). Data and geometries distributed with egs_brachy are described in sections 8. and 9., respectively. Section 10. provides an overview of a suite of example input files distributed with egs_brachy, and these provide a good starting point for egs_brachy users. Finally, section 11. provides practical guidance on simulation workflow, geometry setup, performance tuning, and validation of results.

The initial release of egs_brachy is v2017.09.15 beta. We will continue to update and maintain egs_brachy, as well as this manual and the html documentation. If you are familiar with git, bug fixes and additional features are welcomed via pull requests on GitHub. Please refer to the EGSnrc Wiki Developers section at https://github.com/nrc-cnrc/EGSnrc/wiki for general recommendations about making pull requests.

Links shown in blue in the pdf file are linked and you will be taken to the link (either a url or section or whatever). Text in magenta is code and only shown to draw attention to it.

When egs_brachy is used for publications, please cite our paper [1]: M. J. P. Chamberland, R. E. P. Taylor, D. W. O. Rogers, and R. M. Thomson, egs_brachy: a versatile and fast Monte Carlo code for brachytherapy, Phys. Med. Biol. **61**, 8214–8231 (2016).

2. Licence

The egs_brachy code (all pieces of code associated with the egs_brachy code system) is copyrighted Rowan Thomson, Dave Rogers, Randle Taylor, and Marc Chamberland. egs_brachy is distributed in the hope that it will be useful, but without any warranty; without even the implied warranty of merchantability or fitness for a particular purpose.

egs_brachy is distributed as free software according to the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option), any later version. See the GNU Affero General Public License for more details, available at http://www.gnu.org/licenses/.

The EGSnrc Code System is needed for egs_brachy, and is distributed at https://github.com/nrc-cnrc/EGSnrc by NRC independently of egs_brachy under its own separate licence, however it too follows the GNU Affero General Public License as published by the Free Software Foundation.

3. Installation

Installing and running egs_brachy requires the CLRP fork of the EGSnrc source code (EGSnrc_CLRP, egs_brachy branch), not the standard NRC distribution. The CLRP fork includes additional egs++ functionality needed by egs_brachy.

Even if you already have EGSnrc installed, a separate install (different EGS_HOME) is recommended; switching between installs means changing EGS_CONFIG, HEN_HOUSE, and EGS_HOME.

3.1. Installing from a release tarball

CLRP publishes release tarballs on GitHub: https://github.com/clrp-code/EGSnrc_CLRP/releases

Download EGSnrc_CLRP-eb-setup-VERSION.tar.gz (bootstrap installer). Release tags are named egs_brachy-VERSION; the VERSION in the tarball filename is the part after that prefix (e.g. tag egs_brachy-2026-06-23-alpha.3 corresponds to EGSnrc_CLRP-eb-setup-2026-06-23-alpha.3.tar.gz).

Extract anywhere and run install (default install tree is $HOME/EGSnrc_CLRP):

```
  tar -xzf EGSnrc_CLRP-eb-setup-VERSION.tar.gz
cd EGSnrc_CLRP-eb-setup-VERSION
./eb-setup.sh install --egs-home $HOME/egs_home/
```

Follow the interactive configure prompts when asked. eb-setup.sh then syncs egs_brachy to $EGS_HOME and writes eb-env.sh in the install tree.

For day-to-day use, from the install tree:

```
cd $HOME/EGSnrc_CLRP
```

```
source ./eb-env.sh
./eb-setup.sh check
```

Offline install or pin a specific release: use `install -from-tarball PATH` with a downloaded `EGSnrc_CLRP-egs_brachy-VERSION.tar.gz`, or prefix the command with `EB_RELEASE_TAG=VERSION` (GitHub tag `egs_brachy-VERSION`). Run `./eb-setup.sh help` for all options.

After install, you can confirm the build with:

```
cd $EGS_HOME/egs_brachy
make test
```

3.2. Git checkout

Developers may work from a git tree instead. Clone `EGSnrc_CLRP`, check out `egs_brachy`, and initialize submodules (`egs_brachy` is at `HEN_HOUSE/user_codes/egs_brachy`):

```
  git clone https://github.com/clrp-code/EGSnrc_CLRP.git
cd EGSnrc_CLRP
git checkout egs_brachy
git submodule update --init --recursive
./HEN_HOUSE/scripts/configure
```

From a bootstrap tarball, `./eb-setup.sh install --git` performs the clone to `--install-dir` (default `$HOME/EGSnrc_CLRP`).

If you already have an NRC `develop`/`main` checkout, add the CLRP remote (`git remote add clrp https://github.com/clrp-code/EGSnrc_CLRP.git`; `git fetch clrp`) and check out `egs_brachy` as above.

3.3. Updating egs_brachy

From a release install tree (default `$HOME/EGSnrc_CLRP`):

```
cd $HOME/EGSnrc_CLRP
source ./eb-env.sh
./eb-setup.sh update
```

`update` downloads the latest non-draft GitHub release and merges it, preserving local `HEN_HOUSE/specs` and `EGS_HOME` layout. When offline, use `update --from-tarball PATH`.

On a git install tree, `./eb-setup.sh update` pulls `EGSnrc_CLRP` and the `egs_brachy` submodule and rebuilds. Use `--strict` or `--stash` if you have local changes.

After an update, run `make test` in `$EGS_HOME/egs_brachy` to confirm the installation.

4. Running egs_brachy

Once EGSnrc and egs_brachy are fully installed, specific simulations can be run as follows, assuming that an appropriate input file, e.g., my_input.egsinp is available on $EGS_HOME/egs_brachy. The assumption is that the .egsinp file is designed to run in pegsless mode.

```
egs_brachy -i my_input
```

or

```
ex egs_brachy my_input pegsless
```

To capture the extensive output to the terminal in either of the above cases, at the end of the command pipe the output to a file by adding > my_input.egslog.

egs_brachy runs in parallel in batch mode but only if a batch submission process is available and set up (controlled via $HEN_HOUSE/scripts/batch_options.$EGS_BATCH_SYSTEM). The user must define EGS_BATCH_SYSTEM as appropriate for their system. There are 5 examples on $HEN_HOUSE/scripts/). Once this is set up, to run in batch use:

```
exb egs_brachy my_input pegsless
```

If the input file is designed to be run using a PEGS4 datafile, then any one of the following methods work:

```
egs_brachy -i my_input -p pegsfile        #interactive
ex egs_brachy my_input pegsfile           #interactive
exb egs_brachy my_input pegsfile          #batch
```

5. Brief description of egs_brachy input files

This section presents a brief overview of the different sections of the input files required by egs_brachy. For egs_brachy details see the on-line html reference manual at https://clrp-code.github.io/egs_brachy/index.html#usage.
For general information related to egs++, see its manual[2] available on-line at http://nrc-cnrc.github.io/EGSnrc/doc/pirs898/index.html .

For convenience, in the following the input filename is considered to be filename.egsinp since the outputs(see below) are linked to filename.

egs_brachy input files have a similar look and feel to many other egs++ applications. The following describes the different input blocks of the input file that are typically required. In all cases the input blocks start and end with :start and:stop delimiter statements *e.g.,*

```
:start some input:
  my_inputs (one or more lines)
:stop some input:
```

The individual parameters are generally input using input keys with format

```
some variable name = input value(s)
```

If there is more than one input value, they are separated by spaces or commas and if they

continue on the next line, there must be a comma at the end of the line to be continued. Any text on a line after a # symbol is a comment.

The order of the input blocks is generally unimportant. However, certain nested input blocks can appear multiple times within another relevant input block. For example, a :start media input: nested input block may appear multiple times within different :start geometry: input blocks and similarly multiple :start geometry: nested input blocks can appear within the :start geometry definition: input block.

5.1. run control

The :start run mode: block of the input file specifies how many histories to run (ncase) and optionally, if needed, nbatch, nchunk, egsdat file format, geometry error limit. nbatch, nchunk are discussed in section 11.9., p 38. PIRS-898 also covers these inputs, including the calculation option that allows, for example, to restart a previously completed simulation.

5.2. run mode

The :start run mode: block specifies which of the 3 possible run modes are to be used: normal, superposition, volume correction only. These are discussed in section 7.3. on p 18.

5.3. media definition

It is generally expected that users will run egs_brachy in pegsless mode and this requires a :start media definition: input block that specifies the values of AE,UE,AP and UP. To facilitate the use of pegsless mode, many materials are defined in the material.dat file distributed with the code (see section 8.1. p 26 and a list of available materials in Table 2). The material data file = input key should point to the local copy of the material.dat file.

In contrast, to run using a .pegs4dat file as input, submit the job using one of the methods shown in section 4. using the pegsfile input. In this mode, no :start media definition: input block is needed but the media names used must match those in the .pegs4dat file.

5.4. geometry definition

Section 7.2. has a general discussion of geometry modelling in egs_brachy. For details about the inputs for specific geometry classes, refer to the EGSnrc C++ class library Report PIRS-898 (2017)[2] available at http://nrc-cnrc.github.io/EGSnrc/doc/pirs898/index.html The :start geometry definition: block can be the most complex part of the input file for egs_brachy. It must define the following three egs_brachy

specific input keys that are required for the geometry definition input block (in addition to the standard egs++ simulation geometry key):

source geometries = this key specifies which geometries define the actual brachytherapy source object. This may be a single geometry name (*e.g.,* when using the egs_glib shim to load one of the seed geometries suppled with egs_brachy, see section 9.1.) or a series of sub-geometry definitions used to compose a single source geometry when defining geometries inline.

phantom geometries = this tells egs_brachy which geometries to score dose in (1 or more phantom geometries are required). Currently 3 geometry types are allowed as phantoms(and many pre-defined phantoms are supplied, see section 9.2.):
EGS_XYZGeometry (library egs_ndgeometry)
EGS_RZGeometry (library egs_rz)
EGS_cSpheres (library egs_spheres)

source envelope geometry: this input is only required for superposition run mode and must name the EGS_ASwitchedEnvelope geometry that contains the sources. Note that the output .3ddose files associated with these libraries have formats described in section 6.2..

In general the :start geometry definition: input block consists of a series of :start geometry: nested blocks. Individual components of the geometry can be specified in separate :start geometry: blocks and then these can be combined in a further :start geometry: block using composite geometries (*e.g.,* egs_genvelope).

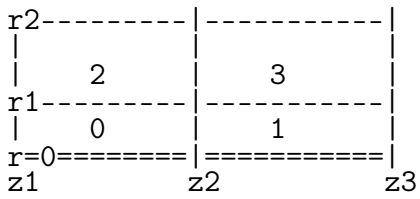
egs_brachy has extended the egs++ package with the egs_glib geometry class that allows the user to insert the contents of an externally defined geometry into an input file, thereby allowing *e.g.,* models for seeds to be distributed and used in multiple input files.

There are many geometry components that are already defined and found in seed_name.geom or phantom_name.geom files on individual subdirectories under $EGS_HOME/egs_brachy/lib/geometry. Examples of such supplied geometries are sources/OncoSeed_6711.geom, phantoms/2.0cmx2.0cmx2.0cm_1mm_xyz_water.geom. For more discussion of the pre-defined and distributed geometries see section 9. These components can have their own :start geometry definition: blocks that are subsequently nested inside a :start geometry: block. These components can be quite complex and one should therefore use as many pre-defined geometries as possible. These predefined geometries use the materials from the material.dat file so it is important to use the pegsless option or ensure that all the materials used are defined in the .pegs4dat file being used..

A typical :start geometry: block will have a name = input key (so it can be referenced elsewhere in the input file) and a library = input key which specifies which type of geometry block this will be, *e.g.,* egs_spheres, egs_cylinders, egs_planes, egs_cones, egs_box, egs_rz and then a series of input keys specific to each library class. Users are urged to carefully check that geometry names, including the ones defined in external files, are unique because there is currently no check in egs++ if geometries are unique (and thus unexpected results may occur if the same name is used for two different geometries).

Within many :start geometry: blocks there will be a further nested block, :start media input: which is used to define the media in each of the local regions. The first input key, media = is used to define, in order from 0 to as many as needed, the materials for this geometry. This is then followed by as many set medium = input keys as needed. There are two input formats possible:
(i) set medium = ireg, imed means ireg is the local region which is assigned medium imed;
(ii) set medium = ireg1, ireg2, imed assigns medium imed to all regions between ireg1 and ireg2.
If there is no explicit assignment for any region, it is then assigned medium 0, *i.e.,* if there is no set medium = input key, every region is assigned medium 0 (the first one listed). (Note that any statement set medium = 0 0 is redundant but harmless.) Final region numbers depend on the overall geometry and how the different elements are combined. It is advisable to check things are as expected using the egs_view code. Vacuum can be assigned by using the egs++ built-in name vacuum.

As an aside: for the egs_rz geometries, the region numbering starts with regions 0 to nz-1 down the central axis of the cylinder, and nz to 2nz-1 down the next radial ring, etc. Note that for nz depth regions one needs to specify nz+1 depths but only nr for the radial regions since the assumption is that r = 0 is included.. For example, for nz=2,nr=2:



A phantom in the egsphant file format, *e.g.,* derived from DICOM-CT data, may be included in the geometry definition via the egs_glib library, as follows:

```
:start geometry:
    name = phantom
    library = egs_glib
    type = egsphant
    egsphant file = lib/geometry/phantoms/egsphant/fileName.egsphant.gz
    density file = lib/media/material.dat
:stop geometry:
```

The density file indicates to egs_brachy the nominal density of each medium in the egsphant, and currently egs_brachy can read these data from a (a) material.dat file, (b) pegs4 data file, or (c) a simple text file of the format:

```
MEDIUM=medium1
RHO=density1
MEDIUM=medium2
RHO=density2
```

Note that voxel-by-voxel densities, defined in the egsphant file, may vary from the nominal densities.

To combine various geometry components there are several classes. For example, the egs_genvelope class (see documentation at http://nrc-cnrc.github.io/EGSnrc/doc/pirs898/classEGS__EnvelopeGeometry.html) allows the insertion of the object specified by the inscribed geometries = input key into another object specified by the base geometry = input key where these keys must be names of one or more of the other geometries specified in the input file. For example:

```
:start geometry:
    name = phantom_with_seed
    library = egs_genvelope
    base geometry = phantom
    inscribed geometries = seed
:stop geometry:
```

Simulation geometries and particles tracks can be viewed using the egs_view application distributed with EGSnrc. A brief overview of egs_view is available in PIRS-898. Instructions to generate and visualize particle tracks are available in the Getting Started with EGSnrc guide.

5.4.1. egs_autoenvelope

A powerful geometry extension developed for egs_brachy is the egs_autoenvelope library. It automatically detects which voxels of the base geometry contain inscribed geometries, *e.g.*, seeds, thereby allowing faster transport (see section 7.2.). This requires a nested block :start region discovery: to define how to do this step.

```
:start region discovery:
       action = discover # discover (default -- ALWAYS use for egs_brachy),
                         # other options for use in other codes are
                               # discover and correct volume,
                               # discover and zero volume
       density of random points (cm^-3) = 1E6  #default is 1E8
     :start shape:
              type = cylinder
              radius = 0.04
              height = 0.45
      :stop shape:
:stop region discovery:
```

For egs_brachy, *always* use action = discover because volume correction inputs are handled separately with their own inputs. The other options of action=discover and correct volume or action=discover and zero volume are not needed for egs_brachy and are included in the egs_autoenvelope class for its potential use in other applications. Note that the density of random points = input key can be lower than that required for accurate volume correction (for which there is a separate input density of random points) – section 5.6..

The egs_autoenvelope library has two geometry types EGS_AEnvelope (default) and EGS_ASwitchedEnvelope. The egs_autoenvelope library must be used with type = EGS_ASwitchedEnvelope for simulations employing run mode = superposition so only

one source is modelled at a time (*e.g.,* for TG-43-like and HDR simulations with example `egsinp` files described in sections 10.6. and 10.9., respectively).

For applicators such as eye plaques, seed positions are often defined in a frame fixed to the applicator. An `egs_autoenvelope` inscribes each seed using those local transforms (typically from an external `include file`), and an outer `egs_gtransformed` geometry wraps the entire autoenvelope assembly to place the applicator and seeds in the simulation frame. The same local seed file and the same applicator pose must be used consistently in the source definition (section 5.5.): local placements in `:start transformations:` and the shared pose in `source coordinate transform`.

5.5. source definition

The `:start source definition:` input block specifies the required info about the radioactivity in the seeds/sources (the geometry of the seed itself is handled in the `geometry definition` input block – section 5.4.). Within this input block there is a nested input block `:start source:` which specifies the charge (0, -1, 1) via an input key `charge =` and must specify the shape of the radioactivity. This can be quite complex and will usually bear considerable similarity to at least part of the geometry specification of the source. For the distributed sources, the relevant files are named `seed_name.shape` as opposed to `seed_name.geom` which defines the overall seed geometry. See `lib/geometry/sources/` for distributed sources which can be used as examples if you need to create your own.

The energy spectrum of the source must be specified in a `:start spectrum:` nested block. Most often, one of 2 types of spectral inputs is used, either a `tabulated spectrum` or `monoenergetic`. The former uses a `spectrum file =` input key to specify the required spectrum (see section 8.2. for a list of those available on `lib/spectra/`). The latter uses a simple `energy =` input key (in MeV).

Next one needs to specify the location of the source(s). This is done in the `:start transformations:` nested block. This can specify many replications of the source via `:start transformation:` blocks. One `:start transformation:` block is required for each seed; `lib/geometry/transformations` gives examples of various arrangements. Note that at least one `:start transformation:` block is needed, even if a null transformation, to tell there is a single source (by default there are no sources). If none is specified, it is assumed there is a single source at the origin but there is a warning message printed. For simple cases in which all sources are placed directly in the simulation frame, the per-source transformations must match those used for geometry inscriptions of the source locations; see section 10.4. for a relevant example `egsinp`. When sources are defined relative to an applicator (section 5.4.1., section 9.3.), the per-seed entries in `:start transformations:` list local placements and must match the inscriptions in the `egs_autoenvelope`; a shared applicator pose is applied separately in geometry (`egs_gtransformed`) and in the source definition (`source coordinate transform`, below).

Optionally, when source placements are defined in a local frame (for example, seed positions relative to an eye plaque) and a separate affine transform places that frame in the simulation, an additional `:start source coordinate transform:` nested block may be

used. It must contain exactly one standard :start transformation: block (translation and/or rotation, in the usual egs++ syntax). At initialization, egs_brachy composes this transform with each entry from :start transformations: so that the world source location is $T_{\text{world},i} = T_{\text{coord}} \cdot T_{\text{seed},i}$. The per-seed entries in :start transformations: are then expressed in the local frame; the coordinate transform maps them to world coordinates for particle transport. This is the source-definition counterpart of the geometry workflow in which an egs_gtransformed envelope wraps an egs_autoenvelope of locally placed seeds: the same local seed-transform file should be referenced for geometry inscriptions and for :start transformations:, and the egs_gtransformed block must use the same affine transform as source coordinate transform. Do not use egs_transformed_source for this workflow. When applied, egs_brachy reports the composition under egs_brachy Source Information in the run log. See the on-line technical reference manual and section 9.3. for eye-plaque examples.

The :start source definition: block must also explicitly specify the simulation source = input key which normally corresponds to the name = key in the :start source: block.

5.6. volume correction

The :start volume correction: input block gives the information required to do a volume correction which basically subtracts the volume of a seed or source from the phantom voxel it is in so that the dose is properly determined. The correction type = input key specifies which type is used:

1. correct: the volume of phantom voxels is properly corrected to account for overlap with source geometries.
2. none: no volume correction is done, which means the nominal volume of each phantom voxel is used when scoring dose.
3. zero dose: if a source geometry overlaps with a phantom voxel, the dose within the voxel is set to zero, i.e., no dose will be scored within this voxel.

The volume is determined using a Monte Carlo method. The density of random points (cm^3) = input key specifies the density of points to use. Values between 1E6 and 1E8 are common, but it depends on the size of the seed/source and the accuracy required. Recall that the dose immediately adjacent to any seed will be very high and is not often of much interest.

The total coverage threshold % = input key specifies the level of voxel coverage by a seed geometry before the voxel is considered totally covered. The threshold is specified as a percentage of the volume of the voxel. If a voxel is covered by more than the threshold (99.9% of its volume covered, by default), then the volume of the voxel is set to 0, and thus, dose in this voxel will always be 0. This avoids having very small voxels with huge uncertainties in their doses due to poor statistics (i.e., very few particles interact in these very small volumes).

Equivalently, if the Monte Carlo volume correction leaves at most (100 − threshold)% of the nominal voxel volume for dose scoring, the corrected volume is set to zero. A lower threshold (e.g. 98) zeroes voxels sooner and is sometimes used for complex applicators such as eye plaques; see `eb_tests/volume_correction/vc.egsinp` and the eye-plaque template `BEBIG16mm-hetero-I125-S06_3rdSeedActive-.egsinp`. This input may be used in either `:start source volume correction:` or `:start extra volume correction:` blocks.

```
:start volume correction:
    :start extra volume correction:
        correction type = correct
        density of random points (cm^-3) = 1E8
        total coverage threshold % = 98   # zero voxels with <= 2% uncorrected volume
        :start shape:
            type = box
            box size = 4
        :stop shape:
    :stop extra volume correction:
:stop volume correction:
```

Within this input block there is a nested block, `:start shape:` which specifies the outer boundary of the seed/source (*e.g.,* using the `boundary.shape` files found in the seed/source definition subdirectories). This shape is then transformed to each of the locations specified for the sources (see previous section). Note that the initial seeds of the random number generator used for the purpose of volume correction or seed discovery are not automatically incremented between jobs when running in parallel. Thus, each parallel job will calculate the exact same volume corrections because they use the same initial seeds. This is the expected and desired behavior.

Alternatively, the simulation can make use of a predetermined set of volume corrections in a `file.volcor` file (see section 5.7. for how to create it). Using this file is controlled by a `:start volume correction from file:` nested block with an input key `phantom file = phantom_name oldfile.volcor` where `phantom_name` is the name of the phantom described in this input and `oldfile.volcor` is the previously calculated corrections file appropriate to this phantom and seed combination.

5.7. scoring options

The `:start scoring options:` input block specifies different scoring options. Scoring options in general are discussed in section 7.4.. For a detailed listing of scoring options, see the on-line html Technical Reference Manual for `egs_brachy` at https://clrp-code.github.io/egs_brachy/index.html#scoringopts.

For scoring dose using the tracklength estimator, only the input keys `muen file =` and `muen for media =` are needed. The former specifies where the μ_{en}/ρ data are stored for the medium the dose is to be scored in (the file `lib/muen/materials_1500keV.muendat` contains a large selection, see Table 3 for a partial list). Any medium in which you wish to score dose needs to be listed in the 'muen for media' key (order unimportant).

If the output from the already-specified volume corrections is required, the output handling method is specified in this scoring options block. Use (a) the input key `output volume correction files for phantoms =` to specify the name of the phantom for which the corrections are required (phantom and name specified elsewhere in the input file) and (b) the key `volume correction file format =` to specify whether `text or gzip` format is desired. Of course, no .volcor output will be formed unless a `:start volume correction:` input block has been specified. The format of the volume correction file is defined in the volume correction section of the technical documentation.

If phase space output is required, a nested `:start phsp scoring:` block is required where the input keys `phsp output directory =, access mode =, print header =, kill after scoring =` are input. The `access mode` can be `write or append` where the former is for a new file and the latter for appending. The `print header` should always be yes for a new file. If the `kill after scoring` input is 'yes', there is no transport done in the phantom, which is efficient for just scoring the phase space for later use, but it could be 'no' to score the dose as well as saving the phase space. Note that phase space scoring does not work for parallel jobs.

Another scoring option is specified in the `:start spectrum scoring:` nested block. The inputs are explained in `ex_single_source_phase_space_and_spectrum_scoring.egsinp`.

```
:start spectrum scoring:
    type = surface count      # options are surface count,
                              # energy weighted surface,
                              # energy fluence in region
    particle type = photon    # photon, electron, positron;
                              # note 'energy fluence in region' type is
                              #       only compatible with photons
    minimum energy = 0.001    # defaults to 0.001 MeV
    maximum energy = 1.00     # defaults to max energy of initial source
    number of bins = 1000     # defaults to 100
    output format = egsnrc    # xmgr (default), csv, egsnrc
                              # egsnrc => format to use as input
                              # spectrum
    file extension =          # (optional)
    # for 'egsnrc' output format only
    egsnrc format mode = 2    # 0, 1, 2 standard EGSnrc options. 2=>line
:stop spectrum scoring:
```

Another useful input key is `dose scaling factor =` which can be used to scale the output dose results, *e.g.,* to being dose in Gy, rather than the dose per effective number of histories, N_{eff} (see discussion related to Table 1 below). An example is in `ex_PeppaBreastHDR192Ir.egsinp` (see section 10.9. below).

The two input keys `score tracklength dose =` and `score energy deposition =` are only needed if their defaults (yes and no respectively) need to be changed. Both options can be used at the same time. The latter scores energy deposited by electrons, so if the source is photons, this is only when a photon interaction occurs and hence is much less efficient. This is referred to as 'interaction scoring' in section 7.4.1.. The outputs are found in the files `filename.phantom.3ddose` for the tracklength scoring and `filename.phantom.edep.3ddose` for the energy deposition scoring.

5.8. variance reduction

The :start variance reduction: is used to turn on some of the variance reduction techniques that are not inbuilt (such as pathlength scoring, see section 7.4.1.).

As discussed in section 7.5.1., it is often advantageous to reuse a particle that escapes from the seed/source for run mode = normal. For cases with more than one seed, this should always be done using a :start particle recycling: nested block that by default only uses the recycled particles once for each seed. But it is further possible to recycle that particle n_r times. The :start variance reduction: input block looks like:

```
:start variance reduction:
    :start particle recycling:
        times to reuse recycled particles = 2
        rotate recycled particles = yes
    :stop particle recycling:
:stop variance reduction:
```

Note that recycling is not compatible with run mode = superposition.

Another type of variance reduction is splitting brems photons when modelling an x-ray source, to split each brems photon uniformly 50 times and optionally to also enhance the brems cross section by a factor of 100 (*i.e.*, use BCSE) *e.g.*,

```
:start variance reduction:
    split bremsstrahlung photons = 50 # uniform bremsstrahlung splitting
    bcse medium = W 100     # brems cross-section enhancement factor
                            # of 100 in the tungsten target
:stop variance reduction:
```

Range rejection is automatically on outside sources and off inside sources (if electrons are being transported at all). This can be controlled within the variance reduction block (see https://clrp-code.github.io/egs_brachy/index.html#rangerejection).

5.9. MC transport parameters

EGSnrc requires a series of transport parameters to be defined. These are specified in a :start MC transport parameter: input block. The easiest approach is to use one of the distributed files found at lib/transport/ by using an include file = input key. These include:

```
high_energy_default
low_energy_default
xray_source
electron_transport_10keV
high_energy_sk_calc
low_energy_sk_calc
```

6. Brief description of egs_brachy outputs

6.1. The .egslog output

The user is encouraged to carefully examine the output to the screen (interactive runs) or to `filename.egslog` when run in batch. Even in interactive mode it is useful to pipe the screen output to `filename.egslog`, *i.e.,* add "> `filename.egslog`" to the end of the execution command. Carefully examining the output is to ensure that the code is calculating what is intended. The output is quite extensive but worth examining. Note that when running interactively, there can be error messages right after the submit statement.

The initial output repeats much of what is in the input file, including a specification of the materials used and the cross sections used. Eventually it gets to a summary of the geometry information; this summary must be very carefully analyzed to ensure no mistakes were made since the required inputs are complex.

There is an explicit list of all the files that were read in as required by the input file.

The starting and incremental time for each batch along with the dose and uncertainty in region 0 are presented as the run progresses. The region the dose is specified in can be changed using the `current result phantom region = phantom_name ireg` key in the `:start scoring options:` input block. This allows the choice of which phantom result to monitor. If the `ireg` entry is not present, it is taken as 0.

Note that the CPU time reported is that for simulating the transport only, the total times are listed later.

When the simulations are finished, `egs_brachy` outputs some summary times and statistics on how many steps etc. It also reports how many geometry errors were encountered.

There are some summary statistics on the energy initiated, escaping the sources and escaping the geometry. Make sure these make sense since they can be a good diagnostic tool.

If a phase space file has been requested, there is a detailed summary of the data in that file. The number of particles in the file is listed. The file size will be 29 bytes per particle so be careful not to run too many histories.

The 20 highest doses are reported along with the volumes they were recorded in. Note that in simulations with seeds, the 20 highest doses will be in those regions immediately surrounding the seeds and have smaller uncertainties than many other doses. The average doses and their uncertainties are reported next.

The next section reports the amount of time spent in various parts of the run. For short runs, the initialization for the run, including creating the cross section data and correcting the volumes can dominate the total time.

To repeat, it is critically important to study the output log file carefully to ensure that the inputs were done properly.

6.2. .3ddose and other outputs

In addition to the .egslog outputs, the main output is in the .3ddose files. The pathlength doses are found in filename.phantom.3ddose and if requested, the energy deposition doses are in filename.phantom.edep.3ddose. This file format is used in other EGSnrc applications such as DOSXYZnrc and there are various tools distributed with EGSnrc for manipulating them (*e.g.,* statdose and dosxyz_show). The format of .3ddose files (from section 12 of the DOSXYZnrc Manual[3]) is:

Row/Block 1: number of voxels in x,y,z directions (*e.g.,* nx, ny, nz)
Row/Block 2: voxel boundaries (cm) in x direction(nx +1 values)
Row/Block 3: voxel boundaries (cm) in y direction (ny +1 values)
Row/Block 4: voxel boundaries (cm) in z direction(nz +1 values)
Row/Block 5: dose values array (nx.ny.nz values)
Row/Block 6: error values array (fractional errors, nx.ny.nz values)

The above format assumes an xyz rectilinear phantom. If the egs_rz cylindrical RZ geometry has been specified (section 5.4.), then the .3ddose file reports the z coordinates in the x slots, the radial coordinates in the y slots and the z slots are unused and shown as one region from -999 to +999. If the egs_spheres spherical geometry has been specified, then the .3ddose file reports the radial component in the x slots and the y and z slots are unused and shown as two regions, each from -999 to +999.

There is also a filename.egsdat file output that contains the information needed to do a restart. This can be a large file and should be discarded once the final run has been done/analyzed. In the :start run control: block one could specify
egsdat file format = gzip in which case these large files will be output in compressed format. This can be important when many parallel runs are being done.

Finally, there is a filename.mederr output file that summarizes the inputs for cross section data.

7. Description of egs_brachy

7.1. Radiation transport modelling

egs_brachy is an EGSnrc [4, 5] application, allowing for both photon and electron transport to be modelled. This means that both electronic sources (*e.g.,* miniature x-ray tubes and beta-emitting eye plaques) and photon (radionuclide) brachytherapy sources may be modelled. While electron transport may be modelled, the relatively low energy of photons from brachytherapy sources means that dose is often well approximated by collision kerma for many brachytherapy scenarios (section 7.4.1.). Thus, often only the simulation of photon transport is necessary and electron transport is not modelled via appropriate choice of the electron cutoff energy (ECUT). All the physics underlying EGSnrc may be modelled, *e.g.,* Rayleigh scattering, bound Compton scattering, photoelectric absorption, and fluorescent photons from atomic relaxations (K, L, M, N shells). As with other EGSnrc applications,

the user may select the photon cross section dataset to use in calculations and specify the raw source spectrum.

7.2. Geometry and source modelling

Geometry modelling is handled by egs++ [6] (manual available at http://nrc-cnrc.github.io/EGSnrc/doc/pirs898/index.html). Using the built-in elementary and composite geometries of egs++, complex phantom, source, and applicator geometries can be modelled. egs_brachy is distributed with a library of pre-defined phantom, applicator, and source model geometries to facilitate ease of use (section 9.) and to give working examples.

egs_brachy supports scoring in rectilinear voxels (through the use of the EGS_XYZGeometry class) and in cylindrical and spherical shells using the EGS_rz and EGS_cSpheres classes, which are additions to the egs++ library developed for egs_brachy. These and other additions (further described below - EGS_AEnvelope, EGS_ConicalShellStackShape, EGS_SphericalShellShape) to egs++ are available as part of the egs_brachy distribution. They will eventually be part of the core EGSnrc distribution but may currently be found at https://github.com/randlet/EGSnrc/tree/feature-geometries. Documentation for the additional classes can be found in the class listing of PIRS-898 included with EGSnrc_CLRP, located at $HEN_HOUSE/doc/src/pirs898-egs++/html/annotated.html.

As brachytherapy treatments may involve many sources, a new egs++ geometry class has been implemented, egs_autoenvelope. This class is a fast envelope geometry used to inscribe one or more copies of another geometry inside a base geometry, such as multiple source geometries inside a phantom. During the geometry initialization, voxels within the phantom containing part of a source geometry are identified ('region discovery'): points inside each source geometry are randomly selected and the phantom regions containing these points are determined, thus generating a list of phantom regions containing part of a source. The time for this process depends on, *e.g.,* the number of sources, voxel sizes, number of voxels, phantom size, density of random points [1]. The list of phantom voxels containing sources is used during particle transport to determine whether the current voxel contains a source; if it does not, then there is no check of the boundaries of source geometries since not checking reduces computation time. See section 10.4. for an example egsinp file demonstrating the use of egs_autoenvelope for a prostate LDR simulation involving 100 seeds.

A subtype of egs_autoenvelope, EGS_ASwitchedEnvelope, is a 'switched' egs_autoenvelope that activates and deactivates inscribed geometries (either in a sequential or arbitrary order) in custom egs++ applications. This geometry class was developed for egs_brachy to investigate the effects of interseed attenuation and scatter, as well as to simulate a source stepping through dwell positions in an HDR treatment. It must be used in conjunction with the 'superposition' run mode (section 7.3.). See section 10.9. for an egsinp example.

Within a source, the distribution of radioactivity is specified by sampling random points from a user-specified shape. Two new egs++ shape classes were developed for egs_brachy and can be used by other egs++ applications. EGS_ConicalShellStackShape and EGS_SphericalShellShape sample random points within a stack of conical shells and within spherical shells, respectively. EGS_SphericalShellShape can also sample points within hemispherical shells and in shells truncated by a conical section, which is useful when defining, *e.g.,* the activity distribution within a beta-emitting eye plaque.

The egsphant file format (also used with the EGSnrc application DOSXYZnrc [3]) may be used to model a a rectilinear phantom derived from CT data; note that 'gzipping' egsphant files can result in compression of these potentially large files by $\sim 99\%$ and these files can be directly used in egs_brachy. CT data in the DICOM format may be converted to the egsphant format using ctcreate (distributed with the EGSnrc code) or similar tools.

7.3. Run modes

egs_brachy has three run modes: normal, superposition, and volume correction only.

The default run mode for simulations is normal, in which one or more brachytherapy sources are modelled. Particles may be initiated within the source with a user-defined spectrum or using phase-space data as a source of particles initialized on the surface of the source (referred to as a 'phase-space source').

The superposition run mode may be used with more than one brachytherapy source and requires the use of the EGS_ASwitchedEnvelope geometry class (section 7.2.). In this mode, one source is 'active' at a time; while particles from this source are being transported through the geometry, all other sources are 'virtual', *i.e.,* photons are transported through the material in which the source is embedded. This run mode is suitable for simulating HDR treatments in which there is only one source present in the geometry at a time and the source steps through dwell positions (example egsinp – section 10.9.). This feature can be applied to LDR simulations to remove interseed effects and investigate interseed attenuation or mock-up TG-43 calculations (example egsinp – section 10.6.).

For simulations using multiple instances of the same source model with either the normal or superposition run modes, sources may be assigned different statistical weights; emitted photons will be weighted accordingly. This feature may be used to model sources of differing air-kerma strengths for LDR treatments or varying dwell times at different source positions for HDR treatments (example egsinp – section 10.9.). The overall normalization for user-specified weights is arbitrary as egs_brachy will divide all weights by the highest weight value such that all weights are ≤ 1. See also section 7.4.1. for details regarding weights and dose calculation.

Dose calculations in egs_brachy typically require the volume (or mass) of the voxel in which energy is deposited (section 7.4.1.). The volume correction only run mode may be used to run the volume correction routines of egs_brachy (see section 7.4.1.), output the results, then quit; no radiation transport is done, so the number of histories specified (ncase) is not actually used. The output of a volume correction only run may be used

as a pre-calculated volume correction file. This is especially useful for repeated simulations or simulations being run on multiple cores in parallel with lengthy volume corrections to perform, *e.g.,* in the case of large applicators: a single `volume correction only` job is run to calculate the volume corrections. Subsequent simulations use the volume correction file, eliminating the redundant, potentially time-consuming calculation of volume corrections. See section 10.8. for a relevant example `egsinp` file.

7.4. Scoring options

7.4.1. Dose

Although `egs_brachy` can model both electron and photon transport (section 7.1.), the low energy of photons from most brachytherapy sources means that the range of secondary electrons is very short [7, 8]. Charged particle equilibrium can often be assumed and collision kerma can be considered equal to absorbed dose for many applications of interest in brachytherapy. By default, collision kerma in voxels is scored using a tracklength estimator (see, *e.g.,* ref [9]):

$$D^j = K^j_{\rm col} = \frac{\sum_i E_i t_i \left(\frac{\mu_{\rm en}}{\rho}\right)_i}{V_j} \tag{1}$$

where D^j and $K^j_{\rm col}$ are the dose and collision kerma in the $j^{\rm th}$ voxel, E_i is the energy of the $i^{\rm th}$ photon crossing the voxel, t_i is the tracklength of that photon in the voxel, $\left(\frac{\mu_{\rm en}}{\rho}\right)_i$ is the mass energy absorption coefficient for energy E_i, and V_j is the volume of the voxel. When a voxel in a scoring phantom is partially occupied by another geometry (source, applicator, etc.), energy deposition is only scored in a portion of the voxel, the volume of which must be used to calculate dose. Voxel volume corrections are performed using a Monte Carlo technique to estimate the volume of the voxel occupied by another geometry.

Scoring collision kerma efficiently using equation (1) requires rapid access to mass energy absorption coefficients. Mass energy absorption coefficients for phantom media in which dose is scored are supplied in a specified external file for a given simulation; these data may be calculated using the EGSnrc application *g*. A sample input file for this *g* application is

```
###############################################################################
#
# Input file for the g application
# Based on the example at nrc-cnrc.github.io/EGSnrc/doc/pirs3100/
#
#  - Energy grid from 1 keV up to 1500 keV with 2000 gridpoints
#  - Calculation type 1 (photon beam only)
#  - Desired precision 0.1%
#
###############################################################################
```

```
media= WATER_0.998
INITIAL RANDOM NO. SEEDS= 97, 33
Number of HISTORIES= 1000000000  # 1e9
precision= 0.001                 # target precision is 0.1%
calculation type= 1
verbose = no

:Start Source Input:
    INCIDENT SPECTRUM=       mono-energy-log-range
    INCIDENT KINETIC ENERGY= 0.001, 1.5, 2000
    Incident Charge=         0
:Stop Source Input:

:Start MC Transport Parameter:
    Global ECUT=                    0.512
    Global PCUT=                    0.001
    Global SMAX=                    1e10
    Spin effects=                   On
    Rayleigh scattering=            On
    Photon cross sections=          xcom
    Pair cross sections=            nrc
    Triplet production=             On
    Bound Compton Scattering=       norej
    Radiative Compton corrections=  On
    Atomic relaxations=             On
    Photoelectron angular sampling= On
    Brems angular sampling=         KM
    Brems cross sections=           nrc
    Pair angular sampling=          KM
    ESTEPE=                         0.25
    XIMAX=                          0.5
    Electron Impact Ionization=     ik
    Skin depth for BCA=             3
    Boundary crossing algorithm=    exact
    Electron-step algorithm=        default
:Stop MC Transport Parameter:

:Start Media Definition:
    ae= 0.512
    ue= 2.511
    ap= 0.001
    up= 2.00
    material data file= $EGS_HOME/egs_brachy/lib/media/material.dat
:Stop Media Definition:
```

Data are stored on the same energy grid as used by the EGSnrc system for materials data and the same lookup algorithms are used as for the cross section data. For accurate calculations, mass energy absorption data must be provided for a reasonable grid of energies; the current default is 2000 points. A file containing mass energy absorption coefficients for various media is distributed with `egs_brachy` (`egs_brachy/lib/muen/materials_1500keV.muendat`) and these coefficients were calculated using the XCOM photon cross section dataset. For consistency the user of `egs_brachy` should use the same data set for photon cross sections. The user may consult the file `egs_brachy/lib/media/material.dat` for the list of elemental compositions. As of September 2026 the `.muendat` file contains the materials listed below in Table 3 for an energy range from 1 keV to 1500 keV. The full dataset is generated automatically using the `muendat-generation` tool from `http://github.com/clrp-code/muendat-generation`.

The user may also choose to employ interaction scoring to compute absorbed dose or collision kerma, depending on the cutoff energy chosen for electron transport. Computation of absorbed dose may be necessary for situations in which the collision kerma approximation is inadequate, *e.g.*, for distances less than 2 mm from the central axis of an ^{192}Ir source (see [10] and references therein) or near other higher-energy brachytherapy sources; in situations where there are significant media heterogeneities; or if the dimensions of scoring voxels are not significantly longer than the secondary electron range.

Whether dose is approximated as collision kerma or absorbed dose is calculated, statistical uncertainties on dose are evaluated using history-by-history statistics [11]

$$s_{D^j} = \sqrt{\frac{1}{N_h - 1}\left[\sum_{i=1}^{N_h}\frac{(D_i^j)^2}{N_h} - \left(\sum_{i=1}^{N_h}\frac{D_i^j}{N_h}\right)^2\right]}. \tag{2}$$

Here, s_{D^j} is the statistical uncertainty per history on the dose D^j deposited in voxel j, while the summation is over the N_h statistically independent starting particles (histories); D_i^j is the dose deposited in voxel j from all particles originating from history i. When particle recycling is used, recycled particles are considered part of the same history as the original particle.

Doses and statistical uncertainties are output within the phantoms specified by the user in the `3ddose` format developed for the BEAMnrc system [3](see section sec-3ddose). The statistical uncertainty on dose in each voxel is output as a fractional uncertainty. Dose in voxel j is output normalized as dose (in Gray) per effective number of histories per source (or source positions, for `superposition` run mode):

$$\bar{D}^j = \sum_{i=1}^{N_h}\frac{D_i^j}{N_{eff}/n_s} \tag{3}$$

The effective number of histories, N_{eff}, is defined in Table 1. In simulations for which starting particles are initiated within the radioactivity distribution within the source (herein referred to as 'ab initio'), N_{eff} is just the number of histories, N_h. When a phase-space source or the particle recycling feature are used (see section 7.5.), N_{eff} is the number of

Table 1

independent histories that would have to be simulated in an ab initio simulation to obtain the same number of scoring particles.

Table 1: Effective number of independent histories, N_{eff}, used for dose normalization (equation (3)), where N_h is the number of initial histories, n_s is the number of sources in the simulation, n_r is the number of times to recycle particles at each source location, N_1 is the number of particles to initialize from the phase-space source, and N_{emit} is the the number of photons emitted from the source from simulation of N_h (initial) histories when generating the phase space data.

Simulation	Description	N_{eff}
(1) Ab initio (default)	All particles initialized within each source	N_h
(2) Ab initio with recycling	Particles initialized within one source; particles escaping that source are recycled n_r times at each of the n_s source locations	$N_h\, n_s\, n_r$
(3) Phase-space source	Initialize $N_1 \leq N_{emit}$ particles from a file containing data for N_{emit} particles from simulation of N_h (initial) histories	$\frac{N_1 N_h}{N_{emit}}$
(4) Phase-space source and recycling	Phase-space source as in (3) with recycling as in (2)	$\frac{N_1 n_s n_r N_h}{N_{emit}}$

Dose may be scored separately for primary, single-scattered, and multiple-scattered particles according to the Primary Scatter Separated (PSS) dose formalism [12] at the user's request. Any particle escaping the source encapsulation is considered a primary until it interacts outside the source. The primary and scatter separated doses are normalized to the source's total radiant energy, which is the sum of the energies of all particles escaping source encapsulation. The formalism is intended for single source characterization. This option should only be used for single source simulations without recycling and with particles initialized within the source (i.e., not from a phase-space source). When using the PSS formalism, the units of the doses are Gray per radiant energy (Gy/R) or simply inverse grams (gˆ-1).

7.4.2. Calculation of absolute dose from doses output by egs_brachy

Calculations of absolute dose for photon sources require values of air-kerma strength. In the following, the initial or starting (time $t = 0$) air-kerma strength of source i, $i = 1, ..., n_s$, is denoted $S_K(i)$; the air-kerma strength per history factor is denoted S_K^{hist} (units: Gy·cm^2/history). Values of S_K^{hist} for source models distributed with egs_brachy are given in the header of each source .geom file (see section 9.1.).

Low-dose rate (LDR) brachytherapy with photon sources:

As described in section 7.3., doses scored by egs_brachy are weighted by the statistical weight of the photon from the particular source. For LDR treatments, source weights should be based on the *starting* ($t = 0$) air-kerma strength of each source, *e.g.,* weights will be input as $S_K(i)$. Then, with the maximum *starting* ($t = 0$) air-kerma strength amongst the n_s

sources denoted $[S_K]_{max}$, egs_brachy normalizes weights internally as $w_i = S_K(i)/[S_K]_{max}$ such that the source with the highest air-kerma strength has weight unity. If all sources have the same air-kerma strength then there is no need to input weights (all sources equally weighted). Then taking $t = 0$ and t_{end} as the treatment start and end times, respectively, the total dose in voxel j is:

$$D^j_{tot} = [S_K]_{max} \frac{\bar{D}^j}{S^{hist}_K} \tau \left(1 - e^{-\lambda t_{end}}\right), \tag{4}$$

where τ is the radionuclide mean lifetime with $\tau = 1/\lambda = t_{1/2}/\ln 2$ where standard notation is used for the decay constant, λ, and half-life, $t_{1/2}$. For a permanent implant, taking $t_{end} \to \infty$,

$$D^j_{tot} = [S_k]_{max} \frac{\bar{D}^j}{S^{hist}_K} \tau \,. \tag{5}$$

For clarity, egs_brachy (by default) outputs the values of $\bar{D}^j$ and it is up to the user to apply the relevant equations for their case either as a post-processing step or using the dose scaling factor input.

High dose rate (HDR) brachytherapy with photon sources:

Assuming the source air-kerma strength is constant over the treatment and that the dwell times are much shorter than the mean (radionuclide) lifetime, then internally the weights are taken as $\Delta t_i/\Delta t_{max}$ where $i = 1, ..., n_s$ enumerate the source dwell positions, Δt_i is the dwell time for source position i, and Δt_{max} is the maximum individual dwell time. However, the user is free to input the dwell times directly as the weights, or the fractional dwell times at each position and egs_brachy internally calculates the required weights. Given these weights, the total dose in voxel j is:

$$D^j_{tot} = [S_K]_{max} \frac{\bar{D}^j}{S^{hist}_K} \Delta t_{max}, \tag{6}$$

where $[S_K]_{max}$ is the starting ($t = 0$) source air-kerma strength and the user will need to know Δt_{max} to use the outputs. As the half-life of ^{192}Ir is 73.8 days and dwell times are often on the order of seconds, these approximations may well be adequate for many applications. However, without these assumptions, the (un-normalized) weights should be $S_K(t_{i,1})\left(e^{-\lambda t_{i,1}} - e^{-\lambda t_{i,2}}\right)/\lambda$ (where $t_{i,1}$ and $t_{i,2}$ are the start and end times, respectively, for dwell position i, $i = 1, ..., n_s$) which will be normalized by egs_brachy via division by the maximum weight over all the source positions, $\max\left\{S_K(t_{i,1})\left(e^{-\lambda t_{i,1}} - e^{-\lambda t_{i,2}}\right)/\lambda \,|\, i = 1, ...n_s\right\}$. The total dose in voxel j is then:

$$D^j_{tot} = \frac{\bar{D}^j}{S^{hist}_K} \max\left\{S_K(t_{1i})(e^{-\lambda t_{i,1}} - e^{-\lambda t_{i,2}})/\lambda \,|\, i = 1, ...n_s\right\}. \tag{7}$$

7.4.3. Phase space

A particle's position, charge, and energy, as well as the direction of its trajectory comprise its phase-space data. egs_brachy can tabulate phase-space data on the surface of a source for all

particles emitted from the source. Optionally, particles can be discarded immediately after having been scored to the phase space to increase the speed of the generation of the phase-space data. Phase-space data are written in the IAEA format [13] which is a generalization of the original BEAMnrc phase space format since it also allows the z position to be included.

7.4.4. Spectrum

egs_brachy has four spectrum scoring options:

1. an absolute count, on the external surface of the source (e.g., on the capsule of brachytherapy seeds), of the particles (photons or charged particles) escaping a source;
2. an energy-weighted spectrum of particles (photons or charged particles) scored on the surface of a source;
3. the photon fluence in a single geometry region; and
4. a photon energy fluence spectrum scored in a voxel.

For the second option, energy is scored when particles are emitted from the surface. For the third option, the photon fluence in a user-specified voxel is calculated as the total photon pathlength in a given energy bin divided by the volume of the region, which is equivalent to the fluence averaged over the voxel volume [14, 15]. The fourth option is calculated similarly to the third option, but the pathlength of the photon is multiplied by its energy.

Scored spectra may be output in a file formatted for plotting with xmgrace, or in the three possible EGSnrc input spectrum 'modes' enumerated as: 0 (histogram in counts per bin); 1 (histogram in counts per MeV); and 2 (line spectrum).

Further details may be found at https://clrp-code.github.io/egs_brachy/index.html#specscoring.

7.5. Features to enhance simulation efficiency

egs_brachy was designed to be a fast MC code for brachytherapy. Geometry handling is designed to be efficient, *e.g.,* by identifying phantom voxels containing part of a source during geometry initialization (section 7.2.). By default, dose is approximated as collision kerma scored using a tracklength estimator (section 7.4.1.), and hence electron transport is not typically needed. Other features to enhance simulation efficiency are discussed in the current section.

7.5.1. Recycling of photons emitted from sources

For simulations of $n_s > 1$ sources of the same model with the normal run mode, the simulation of photons within sources need not be repeated for every history. Using the particle recycling feature, the first source in the simulation acts as a particle generator such that

particles initiated in this source are tracked until they are either absorbed within it or they escape the source encapsulation. If the particle does not escape the source, the history counter is incremented by one and another particle is initiated in the first source. This process is repeated until a particle escapes the first source. The escaping particle is then translated and initiated for each source at the same relative position as it escaped the first source or rotated by a random angle about the source axis before being re-initiated at each source location. This is called recycling. Each generated particle may be recycled more than once (n_r times) at each source location; in this case, the rotation is not optional and particles are rotated by a random angle about the source axis before each reuse so they don't both go through the same voxels.

Recycled particles from the same primary history are not statistically independent. The computation of statistical uncertainties accounts for correlations between particles: the history counter is incremented by one for each set (i.e., $n_s n_r$) of recycled particles to preserve the history-by-history statistics (Table 1, p22).

In general when using particle recycling with $n_r = 1$ (*i.e.,* when using multiple sources), rotating leads to slightly less efficient simulations than recycling particles without rotation since photons are less likely to be in the same voxel if they all start in the same direction from different seeds. The COMS eye plaque simulations are more efficient even when $n_r = 1$ if particles are rotated prior to being recycled at each source location because of the particular spatial arrangement of seeds within the plaque; as the statistical uncertainty accounts for correlations between recycled particles from the same primary history, when the rotation option is not on, several of these photons deposit dose in the same voxels, resulting in an increase in computing time with little improvement in statistical uncertainty. Prostate and breast LDR simulations are generally more efficient when rotation is off as photons from the same history are less likely to deposit dose in the same voxels. In general, recycling particles more than once at each source location does not lead to further efficiency gains. See Table 6 of ref[1] for specific examples.

7.5.2. Use of phase-space source

The user specifies the number of particles, N_1, to be initiated from the phase-space data. If this number exceeds the number of particles for which data exist in the phase-space file (N_{emit}), then `egs_brachy` generates a warning that the phase-space file is being used more than once. This situation may lead to biasing as the calculation of statistical uncertainties will not account for the fact that particles are not all independent. If the user wishes to simulate more starting particles than there are data for in the phase space (i.e., $N_1 > N_{emit}$), particle recycling should be used in conjunction with the phase-space source since this will properly account for correlations between particles from the same primary history (Table 1).

7.5.3. Features for electronic brachytherapy sources

In miniature x-ray tube sources, electrons set in motion impinge on a bremsstrahlung target. As discussed by Ref. [16], the probability of bremsstrahlung emission from low-energy elec-

trons decelerating in target material is very small, resulting in a large fraction of CPU time spent tracking electrons that stop without giving off photons and hence create inefficient simulations. egs_brachy users have the option to use bremsstrahlung cross section enhancement (BCSE), uniform bremsstrahlung splitting (UBS), and Russian roulette to enhance simulation efficiency. With the BCSE feature, the user specifies a factor f_{enh} by which to scale up the bremsstrahlung production cross section in the target material. Using UBS, the user specifies N_{split} and each generated bremsstrahlung photon is split into N_{split} photons. Russian roulette is automatically activated if BCSE or UBS are used, and then secondary charged particles produced have a survival probability of $1/(f_{enh}N_{split})$. The original BCSE paper [16] suggests optimal values for the required parameters, $f_{enh} = 500$ and $N_{split} = 100$, for the $(1\text{ mm})^3$ array of voxels they considered (note that optimal values will depend on the simulation under consideration) [1].

8. Data distributed with egs_brachy

8.1. Material data definitions distributed with egs_brachy

The user may opt to run the egs_brachy application using the traditional .pegs4dat data file or in pegsless mode. For the latter, the materials' properties are defined in the file egs_brachy/lib/media/material.dat. The user can easily add any material needed to the egs_brachy/lib/media/material.dat file. Two typical definitions are:

```
          medium = AIR_TG43
          rho = 0.0012
          elements = H, C, N, O, AR
          mass fractions = 0.0732, 0.0123, 75.0325, 23.6077, 1.2743
          gas pressure = 1.0
          bremsstrahlung correction = NRC
-------------------------------------------------------------------
          medium = WATER_0.998
          rho = 0.998
          elements = H, O
          number of atoms = 2, 1
          bremsstrahlung correction = NRC
```

Note that, in our experience, using the traditional .pegs4dat data file results in slightly faster total calculation times for simulations involving many media. For simulations with many histories and relatively long calculation times, the time saved using the traditional .pegs4dat data file is probably negligible; however, it may be beneficial to run with the .pegs4dat data file for sub-minute calculation times of simulations involving many media.

As distributed, the file egs_brachy/lib/media/material.dat contains the definitions of the materials in Table 2. A material definition is required for all materials in which particles are being transported. In addition to the information in these files, the .egsinp file must contain information on the upper and lower energy thresholds for creation of charged particles

(AE,UE) and photons (AP,UP) in the :start media definition: block of the .egsinp file (see section 5.3. above). Note that the default photon cross sections are those same defaults used by EGSnrc, in particular the xcom-based cross sections. This can be changed in the input file (to e.g. mcdf-xcom or mcdf-epdl) but care must be taken to then recalculate the the mass energy absorption coefficients used by egs_brachy for that particular photon cross section choice.

As distributed the file egs_brachy/lib/muen/materials_1500keV.muendat contains the mass energy absorption coefficients for those media in which the dose is typically scored using pathlength scoring. Table 3 lists the available materials in this file. The current recommended dataset was created using the XCOM photon cross sections with radiative Compton corrections. Not all materials in the material.dat file are listed below.

Table 2: As distributed, the file egs_brachy/lib/media/material.dat contains the following material definitions, as well as others. The compositions of each material can be seen in this file and are given in the reference manual (https://clrp-code.github.io/egs_brachy/md_media.html)

50GLAND-50ADIPOSE	EXTERNAL_1.00	Poly_Best2335_0.5gpcm
ACRYLIC	EYELENS_ICRU46	Poly_for_BestPd103_1gpcm
ADIPOSE1_WW86	EYES_PHANT	Pt70Ir30
ADIPOSE2_WW86	F_SOFT_TISSUE_ICRU46	Pt75Ir25
ADIPOSE3_WW86	GITRACT_ICRU46	Pt90Ir10
ADIPOSE_H79	GLAND1_WW86	Pyrex_2.4
ADIPOSE_PHANT	GLAND2_WW86	Pyrex_2.4_Cs
ADV_PD_POLY	GLAND2_WW98	RECTUM_ICRP23
AGAR	GLAND3_WW86	RED_MARROW_WW86
AIR_0.0012	GLAND_H79	RIBS_1.92
AIR_PHANT	GLAND_PHANT	RIBS_10_WH87
AIR_TG43	Graphite2.26	RIBS_2_6_WH87
AIR_TG43_Dry	H7C5O3	SILASTIC
AIR_TG43_LD	HEART1_WW86	SILASTIC_BEBIG
AQUEOUS_MAR	HEART2_WW86	SKIN1_WW86
AgBrAgI_6.20	HEART3_WW86	SKIN2_WW86
AgClAgI	HEART_1.05	SKIN3_WW86
AgI	IPlant_active	SKIN_1.09
AgI_6.003	Ir70Pt30	SKIN_H79
Al2O3	KOVAR	SKIN_PHANT
AlN_Y2O3_0.05_3.26	LEAD_GLASS	SOFT_BONE_ICRU44
AlSilicate	LENS_ICRU	SPONGIOSA_WW86
Alumina_2.88	LUNGS_0.26	SdVBC24H24
Au80Cu20	LUNG_UNITRHO	SiO2
BRAIN_PHANT	MANDIBLE_PHANT	TEETH_PHANT
BRAIN_WMATTER_WW1986	MINERALBONE_PHANT	TISSUE_ICRU33
C12H10O3	MODULAY	Ti10W90
C12H18NCl	MODULAY_BEBIG	Ti44.4Ni55.6
C64H95N3O18	MUSCLE1_WW86	Ti_grade2
C85.7H14.3	MUSCLE2_WW86	URETHRA_WW86
CALCIFICATION	MUSCLE3_WW86	URINARY_BLADDER
CALCIFICATION_ICRU46	MUSCLE_PHANT	URINARY_BLADDER_EMPTY
CARTILAGE_PHANT	M_SOFT_TISSUE_ICRU46	URINARY_BLADDER_FULL
CARTILAGE_WW86	PD(50)_C64H95N3O18(50)	URINE
CORTICAL_BONE_WW86	PEEK	VITREOUS_TUMRAM_MAR
CRANIUM_PHANT	PMMA	WATER_0.998
CS10_Organic_polymer	PROSTATE_ICRP23	WATER_1.000
CS10_Organpoly_Pd	PROSTATE_WW86	YELLOW_MARROW_WW86
CS10_polymer	PTV_1.02	Yb169
CS2O_1.47	Pollucite	melanoma1_maughan
DENSIMET_D176	PolySty	quartz_2.21
EXTERNAL_1.00	Poly_Best2335_0.5gpcm	

Table 3: The file egs_brachy/lib/muen/materials_1500keV.muendat contains the mass energy absorption coefficients for the following materials and more. A complete list may be found at https://clrp-code.github.io/egs_brachy/md_media.html

50GLAND-50ADIPOSE	GLAND2_WW86	RECTUM_ICRP23
ADIPOSE1_WW86	GLAND2_WW98	RED_MARROW_WW86
ADIPOSE2_WW86	GLAND3_WW86	RIBS_1.92
ADIPOSE3_WW86	GLAND_H79	RIBS_10_WH87
ADIPOSE_H79	HEART1_WW86	RIBS_2_6_WH87
AGAR	HEART2_WW86	SKIN1_WW86
AIR_0.0012	HEART3_WW86	SKIN2_WW86
AIR_PHANT	HEART_1.05	SKIN3_WW86
AIR_TG43	HEART_BLOODFILLED_WW86	SKIN_1.09
AIR_TG43_LD	LUNGS_0.26	SKIN_H79
CALCIFICATION	LUNG_BLOODFILLED_WW86	SOFT_BONE_ICRU44
CALCIFICATION_ICRU46	LUNG_UNITRHO	SPONGIOSA_WW86
CARTILAGE_WW86	MUSCLE1_WW86	TISSUE_ICRU33
CORTICAL_BONE_WW86	MUSCLE2_WW86	URETHRA_WW86
EXTERNAL_1.00	MUSCLE3_WW86	URINARY_BLADDER
EYELENS_ICRU46	M_SOFT_TISSUE_ICRU46	URINARY_BLADDER_EMPTY
F_SOFT_TISSUE_ICRU46	PROSTATE_ICRP23	URINARY_BLADDER_FULL
GITRACT_ICRU46	PROSTATE_WW86	URINE
GLAND1_WW86	PTV_1.02	WATER_0.998
GLAND2_WW86	RECTUM_ICRP23	WATER_1.000

8.2. Initial radionuclide spectra

egs_brachy is distributed with initial spectra for a variety of radionuclides as shown in Table 4. They are read in via a spectrum file = input key in the :start spectrum: nested block. These spectra are in the standard ensrc format used by many EGSnrc applications, *viz*:

```
TITLE      spectrum title  (80 char)
NENSRC, ENMIN, MODE (on 1 line)
   NENSRC  # energy bins in spec. histogram
   ENMIN   lower energy of first bin
   MODE    =0 => cts/bin; =1  => cts/MeV; =2 => line spectra
ENSRCD(I),SRCPDF(I)  I=1,NENSRC pair/line
  -modes 0 & 1: top of energy bin and probability of initial particle being
   in this bin.  Probability does not need to be normalised.
  -mode 2: energy of spectral line and its probability.
```

Table 4: Spectra files available in the egs_brachy distribution for different radionuclides. Found on $EGS_HOME/egs_brachy/lib/spectra. Photon only spectra unless otherwise stated. If there are more than two spectra for a radionuclide, the recommended spectrum is indicated by an asterix (∗) after the file name.

Nuclide	Label(.spectrum)	Ref.	Comments
^{60}Co	bareco60_line		
^{103}Pd	Pd103_NNDC_2.6_line ∗	[17]	Based on NNDC 2000 evaluation
^{103}Pd	Pd103_TG43	[18]	Spectrum above fits meas. better.
^{106}Rh	Rh106_ICRU72_line	[19]	β spectrum
^{125}I	I125_NCRP_line ∗	[20, 17]	Fits meas. better than TG43[a]; used by BIPM.
^{125}I	I125_TG43	[18]	Above spectrum is better.
^{131}Cs	Cs131_NNDC_2.6_line	[21]	Khazov et al. (2006).
^{137}Cs	Cs137_NNDC_2.6_line	[21]	Browne & Tuli, Nucl Data Sh. 108(2007)2173.
^{169}Yb	Yb169_NNDC_2.6_line	[21]	Baglin (2008)
^{192}Ir	Ir192_NNDC	[21]	Photons only.
^{192}Ir	Ir192_NNDC_2.6_line ∗	[21]	Baglin (2012) beta- and EC decays.
^{192}Ir	Ir192_bare_1993	[22]	Duchemin and Coursol (1993)

[a] Only significant difference from TG-43 spectrum is 31 keV peak(s).

9. Geometries distributed with egs_brachy

This section provides some brief details regarding some source, phantom, and applicator geometries distributed with egs_brachy. To get a complete list of available geometries, cd $EGS_HOME/egs_brachy/lib/geometry and explore or issue ls -l * . The file names should be self-explanatory. For some geometries, the use of egs_view is useful for seeing the geometry.

9.1. Sources

The source geometry library distributed with `egs_brachy` comprises the models documented in the CLRP TG-43 parameter database version 2 papers: Ref. [23] (40 low-energy LDR sources: 17 ^{103}Pd, 22 ^{125}I, and 1 ^{131}Cs) and Ref. [24] (33 high-energy sources: 1 ^{169}Yb, 23 ^{192}Ir, 5 ^{137}Cs, and 4 ^{60}Co). This includes the three sources benchmarked in the original `egs_brachy` paper [1], ^{103}Pd TheraSeed 200 (Theragenics), ^{125}I OncoSeed 6711 (Amersham), and ^{192}Ir microSelectron-v2 HDR (Nucletron), and the generic ^{192}Ir source developed by the joint AAPM-ASTRO-ABG Working Group on MBDCA (`MBDCA-WG`)[25]. Source models are organized under `lib/geometry/sources/` by radionuclide and commercial model name.

Note that there are two versions of some seed geometries. This occurs when there is a 'hole' at the end of the seed (*e.g.,* TheraSeed 200). In one model this region is filled with air and should be used for in-air calculations. In the other model, this region is filled with water for use when doing calculations in a water phantom. If used in another phantom material, strictly speaking the region should be filled with that material, but since these regions are so small, little difference should be found by leaving them filled with water.

Dose-rate constants (DRCs) for source models distributed with `egs_brachy` are tabulated in the CLRP TG-43 parameter database papers: Ref. [23] for low-energy (^{103}Pd, ^{125}I, and ^{131}Cs) sources and Ref. [24] for high-energy (^{169}Yb, ^{192}Ir, ^{137}Cs, and ^{60}Co) sources. The air-kerma strength per history factor, S_K^{hist}, for each distributed source model is recorded on the `# air kerma:` line in the header of the corresponding `.geom` file under `lib/geometry/sources/`. For example, the first lines of `Cs131_LDR/Proxcelan_CS1_rev2/Proxcelan_CS1_rev2.geom` read:

```
# seed name: Cs131- Proxcelan_CS1_rev2
# air kerma: 3.7155E-14
```

For low-energy sources, S_K^{hist} was determined in the scoring geometry approximating the WAFAC at NIST and then corrected to what the S_K value would be on-axis assuming the source were isotropic using equation 9 in Ref. [26] or the closed form expression in equation 3 of Ref. [17] (with L=0).

9.2. Phantoms

Diverse phantoms are distributed with `egs_brachy` (see `/lib/geometry/phantoms/`); a small subset include:

- `lib/geometry/10.0cmx10.0cmx10.0cm_2mm_xyz_water.geom`: all-water cubic phantom spanning -5.0 cm$< x, y, z <$5.0 cm consisting of a $50 \times 50 \times 50$ array of (0.2 cm)^3 voxels.
- `lib/geometry/10.1cmx10.1cmx10.1cm_1mm_xyz_water.geom`: all-water cubic phantom spanning -5.05 cm$< x, y, z <$5.05 cm consisting of (0.1 cm)^3 voxels.

- `lib/geometry/30cm_0.1mm_sph_water.geom`: all-water spherical phantom with outer radius 15 cm with 0.01 cm thick shells for radii$\leq$ 1.015 cm.
- `lib/geometry/30cmx30cm_r101xz203_0.1mm_rz_water.geom`: all-water cylindrical phantom with total length 30 cm and outer radius 30 cm with 0.01 cm thick cylindrical shells for $0 \leq r \leq 1.015$ cm and -1.015 cm $\leq z \leq 1.015$ cm.
- `PeppaBreastHDR192Ir_MBDCA-WG.egsphant.gz`: Derived from DICOM-CT data for the breast phantom in case 3 of Peppa *et al.* [27].

9.3. Eye plaques

The eye-plaque geometry library distributed with `egs_brachy` comprises the models documented in the CLRP eye plaque brachytherapy database version 2 (CLRP_EPv2) paper, Ref. [28]. Models are organized under `lib/geometry/eye_plaques/` and include eight COMS plaques (10–24 mm), five BEBIG plaques (12–20 mm), and four representative plaque designs (stainless steel, short-lip acrylic/silastic, no-lip silastic, and thin acrylic), validated for ^{103}Pd, ^{125}I, and ^{131}Cs photon-emitting seeds.

As for brachytherapy seeds, some plaque models are distributed in pairs: a standard geometry and a companion `*_water.geom` file in which air gaps are filled with water for calculations in liquid or soft-tissue phantoms. Sample eye-plaque input files are distributed under `lib/geometry/eye_plaques/sample eye plaque input template files/`, and predefined seed-position transformations for COMS plaques are available under `lib/geometry/transformations/eye_plaques/`. For simulations in which seed positions are defined relative to the plaque, geometry and source definition both use a split transform: an `egs_autoenvelope` inscribes seeds with local transforms from a shared `include file`, an outer `egs_gtransformed` geometry carries the plaque pose, and the source definition lists the same local placements in `:start transformations:` with the same pose in the optional `source coordinate transform` block (section 5.5.). A complete worked example is `COMS16mm-I125-6711-hetero-splitTransforms.egsinp` in the sample template directory; it references the shared local seed file `lib/geometry/transformations/eye_plaques/COMS16mm` and the paired pose file `lib/geometry/transformations/eye_plaques/COMS16mm_example_pose`. Dosimetric data for the distributed models are tabulated at https://physics.carleton.ca/clrp/eye_plaque_v2 (DOI http://doi.org/10.22215/clrp/EPv2). A complete list of plaque `.geom` files, sample inputs, and transformation sets is given in the on-line html technical reference manual at https://clrp-code.github.io/egs_brachy/md_geom.html#egs_brachyplaques.

9.4. Applicators

Applicators included in the `egs_brachy` distribution are found in the directory `lib/geometry/applicators`:

- `lib/geometry/applicators/TG186`
  Generic TG-186 Ir-192 shielded applicator[29] developed by members of the joint AAPM-ESTRO-ABG working group on MBDCA, design based on three commercially available gynecological applicators. See section 10.8..

10. Example input files distributed with egs_brachy

A suite of example input files is distributed with `egs_brachy` (found in `lib/examples`). These provide a good starting point for users new to `egs_brachy` and are described in the following subsections. Note that the input files (plus results) in the test suite (found in `egs_brachy/tests`) are also useful.

In these input files, various references to other files are given relative to `$EGS_HOME/egs_brachy`. This will work when the examples are being run interactively. However, when running in batch mode, the complete path to these files must be used, so wherever you see `lib/geometry(etc)` in the example input files, insert `$EGS_HOME/egs_brachy/` immediately before `lib` where **you insert** the full path on your system for `$EGS_HOME`.

These examples almost all use the `pegsless` form of input for cross section data. The example in section 10.5. uses a `.pegs4dat` file for input.

When starting to learn `egs_brachy`, the examples should be studied in the order presented here in the sense that the earlier examples have somewhat more in-line commenting that is not repeated in the later examples.

10.1. ex_single_source.egsinp

A single LDR Pd-103 TheraSeed200 seed in a 30x30x30 cm^3 water phantom; dose is scored in a 2x2x2 cm^3 volume made up of (1 mm)^3 voxels.

10.2. ex_single_source_phase_space_and_spectrum_scoring.egsinp

A single Pd-103 TheraSeed200 seed is modelled in a water phantom; the phase space and spectrum of photons emitted from the source are scored.

10.3. ex_single_source_from_phase_space.egsinp

Same simulation as `ex_single_source.egsinp` (section 10.1.) except that particles are initialized on the source surface using the phase space generated using `ex_single_source_phase_space_scoring.egsinp` (section 10.2.) – note that that the phase space file must be copied into the directory `lib/phsp/` (that the user must create if it does not already exist). This example also makes use of recycling and rotating the recycled particles.

10.4. ex_prostate_permanent_implant.egsinp

A phantom of prostate-tissue material contains 100 Amersham OncoSeed 6711 seeds; dose is scored in a rectangular 'PTV' 3.4x2.8x3.8 cm^3 in (2 mm)^3 voxels. Includes use of `egs_autoenvelope` involving multiple copies of the same source in different locations, as well as particle recycling.

10.5. ex_prostate_permanent_implant_pegs4.egsinp

Same simulation as `ex_prostate_permanent_implant.egsinp` (section 10.4.) except that it uses an input `.pegs4dat` file rather than the input for a pegsless run normally found in a `:start media definition:` block. The specified `.pegs4dat` file must contain all the media used by the inserted (geometry) files, in this case `Ti, AIR_TG43, AgBrAgI_6.20, Ag, PROSTATE_WW86`. The `.pegs4dat` file is found in: `$HEN_HOUSE/pegs4/data/brachy_xcom_1.5MeV.pegs4dat`.

10.6. ex_prostate_permanent_implant_TG43.egsinp

A 'TG43' version of the simulation defined in `ex_prostate_permanent_implant.egsinp` (section 10.4.): 100 LDR Amersham OncoSeed 6711 seeds; dose is scored in a rectangular 'PTV' 3.4x2.8x3.8 cm^3 in (2 mm)^3 voxels. The phantom and scoring voxels are water, and there are no interseed effects through the use of the `run mode = superposition` and EGS_ASwitchedEnvelope option for the `egs_autoenvelope` geometry. Note that there is no particle recycling for this file as it is not compatible with `run mode = superposition`.

10.7. ex_mbdca-wg_center.egsinp, ex_mbdca-wg_center_water_only.egsinp, and ex_mbdca-wg_x7cm.egsinp

Test cases of the MBDCA-WG: single MBDCA-WG generic ^{192}Ir source in phantoms as described by Ballester *et al.* [25]:

- `ex_mbdca-wg_center.egsinp`: Single source at the centre of a 20.1 cmx20.1 cmx20.1 cm water phantom comprised of (1 mm)^3 voxels, surrounded by air
- `ex_mbdca-wg_center_water_only.egsinp`: Single source at the centre of a water phantom (no surrounding air): (1 mm)^3 scoring voxels span 20.1 cmx20.1 cmx20.1 cm and total phantom is (50 cm)^3.
- `ex_mbdca-wg_x7cm.egsinp`: Same as previous but source is displaced 7 cm in the x-direction.

10.8. ex_hdr_source_with_applicator.egsinp and ex_hdr_source_with_applicator_create_volume_correction.egsinp

These examples model the generic MBDCA working group Ir-192 source with generic shielded applicator [25, 29]. As the volume occupied by the applicator is relatively large, the volume correction can take several minutes; for parallel runs, the initialization time can be greatly reduced by pre-computing volume correction data using `ex_hdr_source_with_applicator_create_volume_correction.egsinp` and then, in subsequent computations using `ex_hdr_source_with_applicator.egsinp`. See discussion at end of section 7.3..

10.9. ex_PeppaBreastHDR192Ir.egsinp

Simulates 'case 3' of Peppa *et al* [27] using an `egsphant` file giving the patient geometry (derived from distributed DICOM-CT data [27]) with MBDCA-WG generic ^{192}Ir source stepping through multiple positions with varying dwell times. The dose scaling factor included normalizes output doses (`3ddose` file) in units of Gy. The source angles are specified.

10.10. ex_xray_sph.egsinp

A simple x-ray source consisting of an isotropic, monoenergetic, electron point source impinging on a thin target tungsten. Electron transport is enabled within the source geometry. Variance reduction techniques are used.

10.11. ex_brem_cyl.egsinp

This is an `egs_brachy` input file for benchmarking against a `DOSRZnrc` x-ray source calculations, *i.e.,* it uses the `egs_rz` geometry. It is a 1 cm^2 parallel beam passing through 1 cm of air and then incident on a 0.0001 cm target of Ti10W90. Tracklength and energy deposition doses are scored in $r = 5.0$ cm / $\Delta z = 0.1$ cm slabs of water extending 4 mm. There is no electron transport in the water so electrons coming from the target deposit all their energy in the first water slab. Note that the tracklength dose in the $z = 0.0$ to 0.1 cm voxel scored by `egs_brachy` will significantly underestimate this dose since it is not captured. The dose in the air will also be significantly underestimated since only photon dose is captured by tracklength scoring.

11. Helpful hints for egs_brachy simulations

11.1. Geometry preview

It is good practice to preview simulation geometries using egs_view. For simulations with run mode = superposition and employing the type = EGS_ASwitchedEnvelope geometry option, only the first source listed will be shown in the preview. If the user wishes to view all source positions simultaneously, then *temporarily* comment out the entry type = EGS_ASwitchedEnvelope to view all sources or source dwell positions. Note, however, that for HDR treatments with source dwell locations in close proximity, it is possible that there may be overlap of sources previewed. Ensure that the type = EGS_ASwitchedEnvelope geometry option is then uncommented for egs_brachy simulations.

11.2. Examining the .egslog output

Before trusting dose results from a simulation, carefully examine the .egslog output (section 6.). Even for interactive runs, it is good practice to redirect screen output to a log file, *e.g.,* by appending > filename.egslog to the execution command. In particular, verify that the geometry summary matches the intended setup, that the list of input files read is correct, and that summary statistics on source escape fractions and energy balance are physically reasonable. The geometry error count (section 11.5.) and the timing breakdown at the end of the log are also useful diagnostics: for short runs, initialization—including volume corrections—can dominate the total CPU time.

11.3. Run modes and workflow

The choice of run mode (section 7.3.) has a significant impact on both simulation time and the physics modelled. For normal mode with multiple LDR seeds, interseed effects are included and particle recycling (section 11.7.) is often appropriate. For HDR treatments with a single source stepping through dwell positions, superposition mode with the EGS_ASwitchedEnvelope geometry option is the usual choice (section 10.9.). The superposition mode can also be applied to LDR simulations to remove interseed effects or to mock up TG-43 calculations (section 10.6.).

Volume corrections can be time-consuming for complex geometries such as eye plaques or large applicators. When the same volume corrections will be reused across multiple simulations, it is good practice to run a single volume correction only job first and reference the resulting pre-calculated correction file in subsequent transport runs (section 7.3., section 10.8.).

11.4. Source and geometry transformations

The locations and orientations of sources must be specified consistently in both the :start source definition: block and the geometry definition for the inscribed sources (section 5.5.). In practice, the simplest approach is to define the transformations once in an external include file and reference that file in both places; see section 10.4. for an example. When seed positions are defined relative to an applicator (section 5.4.1., section 9.3.), both geometry and source definition split the transform: local seed placements in the egs_autoenvelope inscriptions and in :start transformations:, with the applicator pose in egs_gtransformed and source coordinate transform respectively; these paired blocks must agree. Mismatched transformations are a common source of incorrect results that may not produce obvious error messages during a run.

11.5. Geometry errors

During particle transport, egs++ calls howfar at each geometry step to determine the distance to the next region boundary. A geometry error is reported when an inconsistency is detected between the particle position and the region returned by the geometry—most often due to finite-precision arithmetic when a particle lies on or very near a boundary. In egs_brachy simulations this commonly arises at interfaces between source and applicator geometries and the surrounding phantom, particularly where inscribed sources overlap voxel boundaries in composite geometries.

When a geometry error occurs, egs_brachy prints a warning along with the last few geometry steps (old region, position, direction, and intended step) and increments a running count. The total number of geometry errors is reported in the .egslog output at the end of the run (section 6.). By default, the geometry error limit input in the :start run control: block is zero, causing the simulation to terminate on the first geometry error. For production runs, the limit should be set explicitly; in practice, a small number of geometry errors can occur even in well-constructed geometries. Based on experience, the limit may be scaled with ncase while keeping the error rate small relative to the number of histories; for simulations of 10^8–10^9 histories, a limit of 2500 is reasonable at 10^9 histories—a fraction of order 10^{-6}, which is negligible for most dose scoring purposes. At other history counts, a proportional limit is appropriate (*e.g.,* 250 at 10^8 histories):

```
:start run control:
    ncase = 1.0e9
    geometry error limit = 2500
:stop run control:
```

If the geometry error count reported at the end of a run is large, or if errors dominate the log output during the run, the geometry should be investigated rather than simply increasing the limit. Preview the geometry in egs_view (previous subsection) and consider the use of wrapper geometries (next subsection) when errors appear to be associated with overlapping source and voxel boundaries.

11.6. Use of a wrapper geometry about sources

A wrapper is an RZ geometry, made of water (or other material of choice appropriate for the simulation/phantom), that is slightly larger than the seed in diameter and in length (*e.g.*, 1 μm larger). A wrapper may be used to reduce geometry errors (section 11.5.) that occur when seed and voxel boundaries overlap. Thus, while not generally necessary, if the user finds that there are many geometry errors due to overlapping boundaries, then wrappers may be defined directly in the input file (see example `egsinp` file described in section 10.9.).

11.7. Particle recycling

For simulations with multiple instances of the same source model in `normal` run mode, particle recycling can substantially improve efficiency (section 7.5., section 7.5.1.). Recycling is *not* compatible with `run mode = superposition` (section 7.5.). The `rotate recycled particles` option should generally be `yes` for COMS eye plaque simulations and `no` for typical prostate and breast LDR implant simulations, for the reasons discussed in section 7.5.1.. Recycling particles more than once at each source location (`times to reuse recycled particles` > 1) rarely provides further efficiency gains.

11.8. Phase-space sources

Phase-space scoring does not work with parallel jobs (section 7.4.). When reusing a phase-space file as a source, if the number of particles requested (`num`) exceeds the number of records in the file (n_{emit}), `egs_brachy` issues a warning and the statistical uncertainties will not account for the lack of independence unless particle recycling is used in conjunction with the phase-space source (section 7.5.2.). Phase-space files require approximately 29 bytes per particle; take care when scoring large numbers of histories to avoid excessively large files (section 6.).

11.9. `nbatch` and `nchunk` options

EGSnrc applications break simulations into `nchunk` 'chunks'. These are useful during parallel runs since the chunking allows for balancing the compute time taken on each computer, even if they have very different speeds. For simple runs with no parallel processing, by default `nchunk=1` whereas for parallel runs, by default `nchunk=10`. For parallel runs the job is divided into `ncase/(npar*nchunk)` 'chunks' of histories where `ncase` is the total number of histories to be done on `npar` machines in parallel. These 'chunks' are then allocated to the various machines when the previous 'chunk' on that machine is completed. In this way faster machines do more 'chunks'.

Within each 'chunk' there are, by default, `nbatch=10` 'batches' that are not required for statistical analysis, but are checkpoints at which time the current results are written to a data file. This can be very useful since these data files can be used to restart or to analyze

partial results if the system stops for any reason during the run. However, for shorter runs with a large number of dose scoring regions, writing these intermediate files can represent a significant part of the run time and setting `nbatch=1` can save considerable time.

If all computers being used have the same processing speed, then even if `nbatch=1`, setting `nchunk=1` can also save some time since the data are written to disk at the end of each chunk(i.e., at the end of its last batch) and having only 1 chunk saves time.

11.10. Source orientation

By default, `egs_brachy` source models are aligned in the positive z-direction, $\hat{k}$, however, the user may wish to model sources with a particular orientation. Supposing that the user knows the source orientation in terms of its direction cosines, *e.g.,* the source orientation is described by a (unit) vector $\vec{v} = \cos a\,\hat{\imath} + \cos b\,\hat{\jmath} + \cos c\,\hat{k}$, one method for specifying source orientation is via three angles (α, β, γ) that are used by `egs++` to determine the rotation matrix. These three angles may be determined from the direction cosines: following the convention used in `egs++`, the rotation matrix is

$$R = R_x(\alpha)R_y(\beta)R_z(\gamma) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & \sin\alpha \\ 0 & -\sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} \cos\beta & 0 & -\sin\beta \\ 0 & 1 & 0 \\ \sin\beta & 0 & \cos\beta \end{bmatrix} \begin{bmatrix} \cos\gamma & \sin\gamma & 0 \\ -\sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{8}$$

With sources aligned along the z-axis and models generally symmetric under rotations about the z-axis, we take $\gamma = 0$ and $R_z(\gamma = 0)$ is the identity matrix. Applying the resulting rotation matrix to the unit vector in the z-direction and setting equal to $\vec{v}$, we obtain:

$$R \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = R_x(\alpha)R_y(\beta) \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -\sin\beta \\ \sin\alpha\cos\beta \\ \cos\alpha\cos\beta \end{bmatrix} = \begin{bmatrix} \cos a \\ \cos b \\ \cos c \end{bmatrix} \tag{9}$$

Hence the angles α, β may be solved via the equations:

$$\alpha = \cos^{-1}\left(\frac{\cos c}{\cos\beta}\right) = \cos^{-1}\left(\frac{\cos c}{\cos\left[\sin^{-1}(-\cos a)\right]}\right), \qquad \beta = \sin^{-1}(-\cos a)\,. \tag{10}$$

The third equation, $\cos b = \sin\alpha\cos\beta$, may serve as a check of results. The user is encouraged to verify that rotations are as expected via visualization in `egs_view`.

See section 10.9. for an example `egsinp` file involving rotated sources.

12. Acknowledgements

This work was supported by the Natural Sciences and Engineering Research Council of Canada, Canada Research Chairs program, and Ministry of Research and Innovation of Ontario. The authors thank Compute/Calcul Canada and the Shared Hierarchical Academic

Research Computing Network (SHARCNET) for access to computing resources. Thanks to individuals who contributed to egs_brachy testing and development: Christian H. Allen, Stephen Deering, Gavin Hurd and Martin Martinov.

References

[1] M. Chamberland, R. E. P. Taylor, D. W. O. Rogers, and R. M. Thomson, egs_brachy: a versatile and fast Monte Carlo code for brachytherapy, Phys. Med. Biol. **61**, 8214 – 8231 (2016).

[2] I. Kawrakow, E. Mainegra-Hing, F. Tessier, R. Townson, and B. R. B. Walters, The EGSnrc C++ class library, Technical Report PIRS–898(2017), National Research Council Canada, Ottawa, Canada. http://nrc-cnrc.github.io/EGSnrc/doc/pirs898/index.html, 2017.

[3] B. R. B. Walters, I. Kawrakow, and D. W. O. Rogers, DOSXYZnrc Users Manual, NRC Report PIRS 794 (rev B) (2005).

[4] I. Kawrakow, Accurate condensed history Monte Carlo simulation of electron transport. I. EGSnrc, the new EGS4 version, Med. Phys. **27**, 485 – 498 (2000).

[5] I. Kawrakow, E. Mainegra-Hing, D. W. O. Rogers, F. Tessier, and B. R. B. Walters, The EGSnrc code system: Monte Carlo simulation of electron and photon transport, Technical Report PIRS-701, National Research Council Canada, 2017.

[6] I. Kawrakow, E. Mainegra-Hing, F. Tessier, and B. R. B. Walters, The EGSnrc C++ class library, Technical Report PIRS–898 (rev A), National Research Council Canada, Ottawa, Canada. http://www.irs.inms.nrc.ca/EGSnrc/PIRS898/, 2009.

[7] L. Beaulieu, A. C. Tedgren, J.-F. Carrier, S. D. Davis, F. Mourtada, M. J. Rivard, R. M. Thomson, F. Verhaegen, T. A. Wareing, and J. F. Williamson, Report of the Task Group 186 on model-based dose calculation methods in brachytherapy beyond the TG-43 formalism: Current status and recommendations for clinical implementation, Med. Phys. **39**, 6208 – 6236 (2012).

[8] R. M. Thomson, Å. Carlsson Tedgren, and J. F. Williamson, On the biological basis for competing macroscopic dose descriptors for kilovoltage dosimetry: cellular dosimetry for brachytherapy and diagnostic radiology, Phys. Med. Biol. **58**, 1123 – 1150 (2013).

[9] J. F. Williamson, Monte Carlo evaluation of kerma at a point for photon transport problems, Med. Phys. **14**, 567 – 576 (1987).

[10] R. E. P. Taylor and D. W. O. Rogers, EGSnrc Monte Carlo calculated dosimetry parameters for ^{192}Ir and ^{169}Yb brachytherapy sources, Med. Phys. **35**, 4933 – 4944 (2008).

[11] B. R. B. Walters, I. Kawrakow, and D. W. O. Rogers, History by history statistical estimators in the BEAM code system, Med. Phys. **29**, 2745 – 2752 (2002).

[12] K. R. Russell, A. K. Carlsson-Tedgren, and A. Ahnesjö, Brachytherapy source characterization for improved dose calculations using primary and scatter dose separation, Med. Phys. **32**, 2739 – 2752 (2005).

[13] R. Capote, R. Jeraj, C.-M. Ma, D. W. O. Rogers, F. Sánchez-Doblado, J. Sempau, J. Seuntjens, and J. V. Siebers, Phase-space Database for External Beam Radiotherapy: Summary Report of a Consultants' Meeting, Technical Report INDC(NDS)-0484, International Nuclear Data Committee, INDC, IAEA, Vienna, Austria, 2006.

[14] A. B. Chilton, A Note on the Fluence Concept, Health Phys. **34**, 715 – 716 (1978).

[15] A. B. Chilton, Further Comments on an Alternative Definition of Fluence, Health Phys. **36**, 637 – 638 (1979).

[16] E. S. M. Ali and D. W. O. Rogers, Efficiency improvements of x-ray simulations in EGSnrc user-codes using Bremsstrahlung Cross Section Enhancement (BCSE), Med. Phys. **34**, 2143 – 2154 (2007).

[17] M. Rodriguez and D. W. O. Rogers, On determining dose rate constants spectroscopically, Med. Phys. **40**, 011713 (10pp) (2013).

[18] M. J. Rivard, B. M. Coursey, L. A. DeWerd, M. S. Huq, G. S. Ibbott, M. G. Mitch, R. Nath, and J. F. Williamson, Update of AAPM Task Group No. 43 Report: A revised AAPM protocol for brachytherapy dose calculations, Med. Phys. **31**, 633 – 674 (2004).

[19] ICRU, Dosimetry of Beta-Rays and Low-Energy Photons for Brachytherapy with Sealed Sources, ICRU Report 72, ICRU, Washington D.C., 2004.

[20] NCRP Report 58, A Handbook of Radioactivity Measurements Procedures, NCRP Publications, 7910 Woodmont Avenue, Bethesda, MD. 20814 USA (1985).

[21] Brookhaven National Laboratory, National Nuclear Data Center, http://www.nndc.bnl.gov/nudat2.

[22] B. Duchemin and N. Coursol, Reevaluation de l'^{192}Ir, Technical Note LPRI/93/018, DAMRI, CEA, France (1993).

[23] H. Safigholi, M. J. P. Chamberland, R. E. P. Taylor, C. H. Allen, M. P. Martinov, D. W. O. Rogers, and R. M. Thomson, Update of the CLRP TG-43 parameter database for low-energy brachytherapy sources, Med. Phys. **47**, 4656 – 4669 (2020).

[24] H. Safigholi, M. J. P. Chamberland, R. E. P. Taylor, M. P. Martinov, D. W. O. Rogers, and R. M. Thomson, Update of the CLRP Monte Carlo TG-43 parameter database for high-energy brachytherapy sources, Med. Phys. **50**, 1928 – 1941 (2023).

[25] F. Ballester *et al.*, A generic high-dose rate ^{192}Ir brachytherapy source for evaluation of model-based dose calculations beyond the TG-43 formalism, Med. Phys. **42**, 3048 – 3062 (2015).

[26] R. E. P. Taylor, G. Yegin, and D. W. O. Rogers, Benchmarking BrachyDose: voxel-based EGSnrc Monte Carlo calculations of TG–43 dosimetry parameters, Med. Phys. **34**, 445 – 457 (2007).

[27] V. Peppa, E. Pantelis, E. Pappas, V. Lahanas, C. Loukas, and P. Papagiannis, A user-oriented procedure for the commissioning and quality assurance testing of treatment planning system dosimetry in high-dose-rate brachytherapy, Brachytherapy **15**, 252–262 (2016).

[28] H. Safigholi, Z. Parsons, S. G. Deering, and R. M. Thomson, Update of the CLRP eye plaque brachytherapy database for photon-emitting sources, Med. Phys. **48**, 3373 – 3383 (2021).

[29] Y. Ma *et al.*, A generic TG-186 shielded applicator for commissioning model-based dose calculation algorithms for hig-dose-rate ^{192}Ir brachytherapy, Med. Phys. **44**, in press (2017).

Index

$S_K(i)$, 20
$\bar{D}^j$, 21
μ_{en}/ρ, 12
n_s, 20, 23
N_h, 20
N_{eff}, 13, 20
n_r, 23
.geom files, 10
    phantom_name, 7
    seed_name, 7
.shape files
    boundary, 12
    seed_name, 10
:start geometry definition:, 6, 7
:start geometry:, 7
:start phsp scoring:, 13
:start region discovery:, 9
:start scoring options:, 12, 15
:start shape:, 11, 12
:start source coordinate transform:, 10
:start source definition:, 11
:start spectrum scoring:, 13
:start transformation:, 10
:start transformations:, 10
:start variance reduction:, 14
:start volume correction from file:, 12
$EGS_BATCH_SYSTEM, 5
`EGS_ASwitchedEnvelope`, 17
`egs_autoenvelope` , 9, 17, 32, 33
`egs_glib`, 7
3ddose files, 13, 16, 20

access mode
    append, 13
    write, 13
action =, 9
AE, 6
air-kerma strength, 18
    per history, 20, 29
AP, 6
applicators distributed, 30
auto envelope, 9

base geometry, 9
base geometry =, 8, 9
batch mode, 5
BCSE, 14, 24
boundary.shape, 11, 12
brachy_xcom_1.5MeV.muendat, 12
brems cross section enhancement, 24

charge =, 10
collision kerma scoring, 23
COMS plaque, 23
correction type =, 11
current result phantom region =, 15
cylindrical geometry
    region numbering, 8

data distributed, 25
density file, 8
density file =, 8
density of random points, 11
density of random points (cmˆ -3) =, 11
density of random points =, 9
description of `egs_brachy` , 16
discover, 9
discover and correct volume, 9
dose
    absolute, 20, 22
dose scaling factor, 33
dose scaling factor =, 13
dose scoring, 19
dose-rate constants, 29
dwell time, 22

efficiency enhancement, 23
egs++, 7
    manual, 5, 17

EGS_AEnvelope, 9, 17
EGS_ASwitchedEnvelope, 9, 32
egs_aswitchedenvelope, 17
egs_autoenvelope, 9, 17
EGS_BATCH_SYSTEM, 5
EGS_ConicalShellStackShape, 17
EGS_cSpheres, 17
egs_genvelope, 9
egs_glib, 7
egs_rz, 8, 16
  example, 33
egs_spheres, 16
EGS_SphericalShellShape, 17
egs_view, 9, 34
EGS_XYZGeometry, 17
egsdat file, 16
  format, 6
egsdat file format =, 16
egslog file, 15
  hints, 34
egsnrc format mode =, 13
egsphant, 18
egsphant file, 8
egsphant file =, 8
electron transport
  example, 33
electronic brachytherapy sources, 24
energy =, 10
example
  input files, 31
  x-ray source, 33
eye plaque modelling, 30
eye plaques distributed, 30

file extension =, 13

geometries distributed, 28
geometry error limit, 6, 35
geometry errors, 35
geometry modelling, 17, 28
geometry preview, 34
glib, 7
gzip, 18

HDR, 33
height =, 9
helpful hints, 34

initial spectra, 10, 28
input blocks, 6
input file examples, 31
input files
  overview, 5
input spectrum
  formats, 23
inscribed geometries, 9
inscribed geometries =, 8, 9
installation
  git checkout, 4
  new install of EGSnrc, 3
  release tarball, 3
  update, 4
interaction scoring, 19
introduction, 1

kerma scoring, 19
kill after scoring =, 13

LDR, 31, 32
lib/geometry, 7
library =, 7–9
licence, 3

material data, 25
  file key, 6
material data file =, 6
material.dat, 6, 8
  file format, 25
material.dat file
  contents, 25
MBDCA, 29, 33
MBDCA-WG, 32
mederr file, 16
media =, 7
media definition, 6

minimum energy =, 13
muen file =, 12
muen for media =, 12
muendat file
contents, 25

name =, 7–9, 11
nbatch, 6
definition, 36
nchunk, 6
definition, 36
nested input blocks, 6
normal run mode, 6, 18
npar
definition, 36
nr, 8
nz, 8

orientation
source, 37
output format
spectra, 13
output format input key re spectrum, 13
output units, 13
output volume correction files for phantoms =, 12
outputs, 15

particle recycling
hints, 36
particle tracks, 9
particle type =, 13
PEGS4
using, 5
pegs4dat file, 7, 25
example of use, 32
pegsless, 5–7, 25
material.dat file, 25
peppa, 36
permanent implant, 21
phantom
breast, 33
phantom geometries =, 7
phantom.3ddose, 13, 16
phantom.edep.3ddose, 13, 16
phantoms, 29
phase space, 22, 31
hints, 36
phase space output, 13
phase-space sources, 24
hints, 36
phsp output directory =, 13
print header =, 13

radiation transport modelling, 16
radionuclide initial spectra, 28
radius =, 9
range rejection inputs, 14
recycle, 32
recycling, 20, 23
definition, 23
hints, 36
input, 14
recycling source particles, 23
rotation matrix, 37
run control, 6
run mode
description, 18
hints, 34
input, 6
volume correction only, 33
running egs_brachy , 5
Russian Roulette, 24

scoring dose, 19
scoring options, 12, 19
seeds with ‘holes’, 29
set medium, 8
set medium =, 7
simulation source =, 11
source coordinate transform, 10
source definition
transformations, 34
source definition inputs, 10

source geometries =, 7
source modelling, 17, 28
source orientation, 37
sources available, 29
spectrum
    initial, 10
    scoring, 22, 31
spectrum file =, 10
superposition, 32
superposition mode, 34
superposition run mode, 6, 18

TG43, 32
TheraSeed 200, 29
timing
    summary, 15
total coverage threshold % =, 11
tracklength estimator, 19
transformations
    consistency, 34
transport parameter input, 14
type =, 8, 9, 13

UBS, 24
UE, 6
uniform brems splitting, 14, 24
UP, 6
update installation, 4

variance reduction inputs, 14
volcor file, 12
volume correction, 33
    input, 11
volume correction input file, 12
volume correction only run mode, 6, 18
    hints, 34
volume correction file format =, 13
VRT inputs, 14

weight, 20
weights, 18, 22
    seed strengths, 20
wrapper geometry, 36
x-ray source example, 33